DROUGHT-AWARE CROP PRODUCTIVITY DECISION-SUPPORT SYSTEM FOR THE SCARCITY ZONE OF MAHARASHTRA

A Thesis

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

by

GOWTHAM S D (CB.AI.U4AID23113)
GUHAN K B (CB.AI.U4AID23114)
VAISHNAVI GUPTA (CB.AI.U4AID23149)
VYSHNAV KUMAR S (CB.AI.U4AID23151)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS (INDIA)

October 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled "**Drought-Aware Crop Productivity Decision-Support System for the Scarcity Zone of Maharashtra**" submitted by **Gowtham S D, Roll. No: CB.AI.U4AID23113**, **Guhan K B, Roll. No: CB.AI.U4AID23114**, **Vaishnavi Gupta, Roll. No: CB.AI.U4AID23149** and **Vyshnav Kumar S, Roll. No: CB.AI.U4AID23151** for the award of the **Degree of Bachelor of Technology** under the **Faculty of Engineering** is a bonafide record of the work carried out by them under my guidance and supervision at "Department of Artificial Intelligence", Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Submitted for the examination held on .....

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Dr. Unnikrishnan
Thesis Advisor,
Assistant Professor (Sr. Gr),
Department of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS

DECLARATION

We, **Gowtham S D**, **Reg. No: CB.AI.U4AID23113**, **Guhan K B**, **Reg. No: CB.AI.U4AID23114**, **Vaishnavi Gupta**, **Reg. No: CB.AI.U4AID23149** and **Vyshnav Kumar S**, **Reg. No: CB.AI.U4AID23151** hereby declare that this thesis entitled "**Drought-Aware Crop Productivity Decision-Support System for the Scarcity Zone of Maharashtra**" is the record of the original work done by us under the guidance of **Dr. Unnikrishnan**, Assistant Professor (Sr. Gr), Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus. To the best of our knowledge, this work has not formed the basis for the award of any degree/diploma/associateship/fellowship or a similar award to any candidate in any University.

Place: **Signature of the Students**
Date:

COUNTERSIGNED

Dr. K.P Soman
Professor and Dean,
Department of Artificial Intelligence
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

Contents

Acknowledgements **v**

List of figures **vii**

List of tables **viii**

Abbreviations **ix**

List of symbols **x**

Abstract **xi**

1 Introduction **1**

1.1 Background 1
1.2 The problem 1
1.3 Aim of the project 2
1.4 Objectives 2
1.5 Scope 3
1.6 The guardrail 3
1.7 Status at the end of the first phase 3
1.8 Organisation of the thesis 3

2 Literature Review **5**

2.1 Introduction 5
2.2 Weather and rainfall forecasting 5
2.2.1 Statistical time-series models 5
2.2.2 Gradient-boosted trees 5
2.2.3 Recurrent neural networks 6
2.2.4 Attention-based models 6
2.2.5 Rainfall as a zero-inflated variable 6
2.3 Crop-yield prediction with machine learning 6
2.4 Drought definition and indices 7
2.5 Knowledge graphs and retrieval-augmented generation 7
2.6 Research gap 8

3 Study Area and Data **9**
3.1 Study area . . . 9
3.2 Data sources . . . 10
3.2.1 Weather data . . . 11
3.2.2 Crop statistics . . . 11
3.2.3 Contingency plans . . . 12
3.2.4 Census data . . . 12
3.3 Data audit and preprocessing . . . 13
3.4 Summary . . . 14

4 System Architecture and Methodology **15**
4.1 Overview . . . 15
4.2 The ten stages . . . 15
4.3 Key design decisions . . . 16
4.4 How a query flows through the system . . . 17
4.5 Technology stack . . . 17

5 Weather Forecasting **19**
5.1 What this stage has to do . . . 19
5.2 Why daily rainfall needs a special model . . . 19
5.3 Evaluation set-up . . . 20
5.3.1 Chronological split . . . 20
5.3.2 Metrics . . . 20
5.4 The hurdle model (main model) . . . 21
5.4.1 Structure . . . 21
5.4.2 Features . . . 21
5.4.3 Training settings . . . 22
5.4.4 Model development: from nowcast to forecast . . . 22
5.4.5 Results . . . 23
5.5 Comparative study . . . 26
5.5.1 LSTM network . . . 26
5.5.2 SARIMA . . . 29
5.5.3 XGBoost . . . 29
5.5.4 Temporal Fusion Transformer . . . 29
5.6 Comparison of results . . . 30
5.6.1 Hurdle model and LSTM under the same protocol . . . 30
5.6.2 All five models as presented at the first review . . . 30
5.7 Why the hurdle model is the main model . . . 31
5.8 Summary . . . 31

6 Knowledge Graph from the Contingency Plans **33**
6.1 Why a knowledge graph . . . 33
6.2 Build pipeline . . . 33
6.3 Extracting the contingency tables . . . 34
6.4 Season labeller . . . 35
6.5 Graph schema . . . 36
6.6 Querying the graph . . . 38
6.6.1 Worked example . . . 38
6.7 Neo4j export . . . 39
6.8 Limitations . . . 39
6.9 Summary . . . 40

7 MLOps Pipeline **41**
7.1 Why MLOps . . . 41
7.2 Architecture . . . 41
7.3 Components . . . 43
7.4 How it works . . . 43
7.4.1 Data versioning . . . 43
7.4.2 Model registry and the champion . . . 43
7.4.3 Serving . . . 44
7.4.4 Automation . . . 44
7.4.5 Monitoring and champion–challenger promotion . . . 44
7.5 Current status and honest limitations . . . 44

8 Conclusion and Scope for further Research **46**
8.1 Conclusion of the first phase . . . 46
8.2 Scope for further research: plan for the second phase . . . 47
8.2.1 Strengthening the forecasting stage . . . 47
8.2.2 Stage 3: season aggregation . . . 47
8.2.3 Stage 4: condition classification . . . 47
8.2.4 Stage 5: yield prediction . . . 47
8.2.5 Stage 6: explanation . . . 48
8.2.6 Stage 7: human impact . . . 48
8.2.7 Stage 8: district comparison . . . 48
8.2.8 Stages 9–10: knowledge graph and guarded GraphRAG . . . 48
8.2.9 Integration, interface and evaluation . . . 49
8.2.10 Timeline . . . 49
8.3 Closing remarks . . . 49

A Configuration, Schema and Interface Details **51**
A.1 Main model: hurdle configuration (params.yaml) . . . 51

A.2 Comparative model: LSTM configuration 51
A.3 Comparative LSTM: per-variable test R^2 for all districts 52
A.4 Knowledge-graph interface 52
A.5 Forecast API 53
A.6 Repository layout 53

References **54**

Acknowledgements

We express our sincere gratitude to our project guide, **Dr. Unnikrishnan**, Department of Artificial Intelligence, for his steady guidance, for pushing us to keep the problem grounded in what farmers and district officials actually need, and for his patience through every review.

We thank **Dr. K.P. Soman**, Professor and Dean, Department of Artificial Intelligence, for providing the academic environment and facilities that made this work possible, and the members of our review panel for their questions and suggestions at the zeroth and first reviews, which shaped the scope of this project.

We also acknowledge the public institutions whose open data this work is built on: NASA's Prediction Of Worldwide Energy Resources (POWER) project for daily weather data, the Directorate of Economics and Statistics, Ministry of Agriculture and Farmers Welfare, for district crop statistics, ICAR-CRIDA and the state agricultural universities for the District Agriculture Contingency Plans, and the Office of the Registrar General and Census Commissioner, India, for Census 2011 data.

Finally, we thank our families and friends for their constant support.

GOWTHAM S D
GUHAN K B
VAISHNAVI GUPTA
VYSHNAV KUMAR S

List of Figures

3.1 Kharif-season rainfall in each district, 2000–2025. Dips such as 2014 and 2018 correspond to known drought years. 10

4.1 System architecture: ten stages in four phases. Green boxes are built; dashed boxes are planned for the second phase. 15

5.1 Daily rainfall in Latur, 2000–2025. Left: about 62% of days are dry. Right: rainy-day amounts are heavily skewed (note the log scale). The other four districts look the same. 20

5.2 The two-stage hurdle model. One gate and one depth model are trained for each district and each horizon $h = 1, 2, 3$ days. 21

5.3 First (same-day) version of the hurdle model, Latur, one-year validation window ($R^2 = 0.94$, RMSE 1.17 mm, MAE 0.51 mm). Because its inputs included the same day's rainfall, it reproduces the observed series almost exactly; it is a nowcast, not a forecast. The corresponding values for the other districts were R^2 0.92 (Ahmednagar), 0.94 (Beed), 0.88 (Osmanabad) and 0.79 (Solapur). . . . 23

5.4 Hurdle model test metrics for each district and forecast horizon. 24

5.5 Rain gate (Hurdle 1) on the test period: overall accuracy, share of rainy days detected, and share of rain forecasts that turned out dry. 24

5.6 Hurdle model: one-day-ahead rainfall forecast against observed rainfall for each district, September 2025 to September 2026. R^2 and RMSE in the panel titles are for the full test period (2022 onward). 25

5.7 Hurdle model, Latur, Kharif 2025: forecasts one and three days ahead against observed rainfall. 26

5.8 Comparative LSTM: training and validation loss for Latur. 27

5.9 Comparative LSTM: test R^2 for every district and variable. 27

5.10 Comparative LSTM, Latur: predicted against observed values on the test set. Temperature and humidity lie close to the diagonal; rainfall forecasts are compressed towards small values. 28

5.11 Comparative LSTM, Latur: three sample 3-day forecasts. Temperature and humidity follow the observed trend; the heavy rain of 5–6 October 2025 is missed. 29

5.12 Daily rainfall forecast skill of the main hurdle model at each horizon, compared with the LSTM. 30

6.1 Number of district-seasons, 2000–2025, labelled with each condition. Mid-season dry spells during vegetative growth are the most common. 36
6.2 The interactive knowledge-graph viewer. Node types and relations can be switched on and off in the sidebar; clicking a node or edge shows its details and source. Actions and season records are hidden by default for readability. 38

7.1 MLOps flow for the forecasting stage 42

List of Tables

2.1 Forecasting studies that informed the model choices 8

3.1 Rainfall characteristics of the five districts, 2000–2025 (NASA POWER daily data) 9
3.2 Data sources used in the project 11
3.3 Crop-season combinations in the DES data (five districts, 2000–01 to 2022–23) . 12
3.4 District profile extracted from the contingency plans 12
3.5 District name variants across sources 13

4.1 Pipeline stages and their status 16

5.1 Hurdle model: rainfall forecast test results (target dates from 2022 onward) . . . 23
5.2 Comparative LSTM: test results per district (2022 onward) 28
5.3 Rainfall test R^2: main hurdle model vs comparative LSTM 30
5.4 Test R^2 of the five models as presented at the first review (individual set-ups) . 31

6.1 Knowledge-graph build steps 34
6.2 Rules used by the season labeller 35
6.3 Node types 36
6.4 Main relation types 37
6.5 Selected Beed recommendations for soybean returned by the graph 39

7.1 MLOps components 43

8.1 Plan for the second phase (eighth semester) 49

A.1 Hurdle model and monitoring settings 51
A.2 LSTM hyper-parameters 52
A.3 Test R^2 by district and variable 52
A.4 FastAPI endpoints 53

Abbreviations

API	–	Application Programming Interface
CI	–	Continuous Integration
DES	–	Directorate of Economics and Statistics
DVC	–	Data Version Control
EDA	–	Exploratory Data Analysis
GraphRAG	–	Graph-based Retrieval-Augmented Generation
ICAR	–	Indian Council of Agricultural Research
CRIDA	–	Central Research Institute for Dryland Agriculture
IMD	–	India Meteorological Department
KG	–	Knowledge Graph
LLM	–	Large Language Model
LSTM	–	Long Short-Term Memory
MAE	–	Mean Absolute Error
MLOps	–	Machine Learning Operations
NASA POWER	–	NASA Prediction Of Worldwide Energy Resources
RAG	–	Retrieval-Augmented Generation
RF	–	Random Forest
RMSE	–	Root Mean Squared Error
SARIMA	–	Seasonal Auto-Regressive Integrated Moving Average
SHAP	–	SHapley Additive exPlanations
SPI	–	Standardised Precipitation Index
TFT	–	Temporal Fusion Transformer
VPD	–	Vapour Pressure Deficit
XGBoost	–	eXtreme Gradient Boosting

List of symbols

y_t	–	Observed value of a weather variable on day t
$\hat{y}_t$	–	Predicted value of a weather variable on day t
$\bar{y}$	–	Mean of the observed values over the test period
L	–	Lookback window length (45 days)
H	–	Forecast horizon (3 days)
R^2	–	Coefficient of determination
$\mathcal{L}_\delta$	–	Huber loss with threshold δ
p	–	Predicted probability of a rainy day (hurdle model)
τ	–	Rain-probability threshold (0.35)
$G = (V, E)$	–	Knowledge graph with node set V and edge set E

Abstract

Five districts in Maharashtra's traditional scarcity zone, Ahmednagar, Beed, Latur, Osmanabad and Solapur, suffer recurring droughts, delayed monsoons and long dry spells. For each of them the government already publishes a District Agriculture Contingency Plan that tells farmers and officials what to do, crop by crop, when the monsoon is late or a dry spell strikes. In practice these plans are 30 to 50 page PDF documents that must be read by hand, and nothing connects them to the weather that is actually unfolding, to the crop yield that can be expected, or to the number of people who will be affected.

This thesis describes a decision-support system that closes this gap. For a chosen district and season (Kharif or Rabi) the system is designed to (i) forecast the weather with a model trained separately for each district, (ii) classify the season's condition against that district's own 26-year history, (iii) predict and explain crop yield with interpretable machine learning, (iv) translate the yield loss into human impact using Census data, and (v) retrieve the official contingency-plan actions from a knowledge graph and have a large language model (LLM) explain them in plain language. A strict guardrail applies: the LLM may only rephrase facts retrieved from the graph and numbers produced by the models, and every recommendation cites the page of the plan it came from.

The work completed in the first phase (seventh semester) covers three of the ten pipeline stages and the operational backbone. The main forecasting model is a per-district two-stage hurdle model for daily rainfall, the variable that defines every drought condition in the plans. Because about 62% of days are dry and rain amounts are highly skewed, the model first predicts whether it will rain and then how much, using gradient-boosted trees and only information available on the forecast day. It forecasts rainfall one to three days ahead with test R^2 of 0.15 to 0.29 at one day, falling to 0.07 to 0.15 at three days. A comparative study of an LSTM network, SARIMA, XGBoost and a Temporal Fusion Transformer supports this choice: the hurdle model gives better next-day rainfall forecasts than the LSTM in all five districts, while the LSTM forecasts smoother variables such as temperature, humidity and dew point well (R^2 0.85 to 0.94). A knowledge graph of 833 nodes and 4,353 edges was built from the five contingency plans, the crop statistics and the weather history, together with a rule-based season labeller and a query interface that returns cited recommendations. An MLOps pipeline with data versioning, a model registry, a forecast API, scheduled data refresh and drift monitoring was also put in place. The thesis closes with the plan for the second phase: season aggregation, condition classification, yield prediction with SHAP explanations, human-impact estimation,

district ranking and the guarded GraphRAG advisory layer.

Chapter 1

Introduction

1.1 Background

Agriculture in India is still largely rainfed, and in rainfed regions a single delayed monsoon or a three-week dry spell at the wrong moment can wipe out a season's harvest. Nowhere is this more visible than in Maharashtra's traditional *scarcity zone*: the Marathwada plateau and the semi-arid rain-shadow belt east of the Western Ghats. The five districts studied in this project, Ahmednagar, Beed, Latur, Osmanabad and Solapur, all lie in this zone. They receive roughly 620 to 760 mm of rain in a year, three-quarters of it between June and September, and the annual total swings by 20 to 25% from one year to the next (Chapter 3). Severe droughts in years such as 2014, 2015 and 2018 caused widespread crop failure and distress in the region [1].

The State and the Indian Council of Agricultural Research (ICAR) already know what farmers should do in such situations. For every district, ICAR's Central Research Institute for Dryland Agriculture (CRIDA), together with the state agricultural universities, has published a *District Agriculture Contingency Plan* [2]. Each plan lists, crop by crop and soil type by soil type, what should be done if the monsoon is delayed by two, four, six or eight weeks, if a dry spell occurs at the vegetative or flowering stage, if the monsoon withdraws early, or if unusual rains, floods or hailstorms occur. The advice is concrete: switch soybean to a pearl millet and pigeon pea intercrop in a 4:2 row ratio, gap-fill to keep 75% plant population, give a protective irrigation with sprinklers, and so on.

1.2 The problem

Despite their value, the contingency plans are hardly used in day-to-day decisions. Three gaps explain why.

1. **The guidance is locked inside PDFs.** Each plan is a 30 to 50 page document with several different table layouts. Finding the action that applies to, say, *soybean on medium-deep black soil in Beed when there is a two-week dry spell at flowering* means reading the document by hand.

2. **Nothing links the guidance to the weather and the expected yield.** A district officer has to decide separately whether the current season counts as a "delayed onset" or a "mid-season drought", how much the yield is likely to fall, and how many people that affects. No tool connects these steps.

3. **Drought is judged against generic thresholds.** A 20% rainfall deficit means very different things in Solapur (mean Kharif rainfall about 520 mm) and in Latur (about 660 mm). A fair judgement compares each season with that district's own history.

1.3 Aim of the project

The aim of this project is to build a decision-support system that, for a chosen district and season, links the weather, the expected crop yield, the human impact and the official guidance in one place, and explains each answer in plain language. In short, it should answer questions such as: *"The monsoon in Beed is running four weeks late this year. What does that mean for the soybean crop, how many people are affected, and what does the official plan say we should do?"*

1.4 Objectives

1. Build weather-forecasting models trained on each district's own history, so that the system does not depend on an external live weather service.

2. Classify each season (normal, dry spell, deficit, drought-like) against the district's own 26-year record, using the definitions written in the contingency plans.

3. Predict crop yield for each district, crop and season with interpretable machine learning and explain which weather factors drove each prediction.

4. Translate predicted yield losses into human impact using Census 2011 population and workforce data, and rank the five districts by priority.

5. Convert the five contingency plans into a structured knowledge graph and retrieve the official actions that match a given district, crop, season and condition, with page citations.

6. Use a large language model only to explain retrieved facts and model outputs in natural language, never to produce numbers or recommendations of its own.

7. Run the forecasting part as a maintained service with data versioning, a model registry, scheduled data refresh and performance monitoring.

1.5 Scope

- **Districts:** Ahmednagar (now officially Ahilyanagar), Beed, Latur, Osmanabad (now Dharashiv) and Solapur.
- **Seasons:** Kharif (monsoon, roughly June to October) and Rabi (post-monsoon, roughly October to March). The Summer/Zaid season was examined in all five plans and found to have negligible cropped area, so it is excluded.
- **Crops modelled:** Kharif Jowar, Soybean, Tur (pigeon pea), Sunflower and Bajra (pearl millet); Rabi Jowar, Sunflower and Gram (chickpea). Cotton, Wheat and Safflower form an optional extended set. Sugarcane (a 12 to 18 month crop) and horticulture crops are not modelling targets, but they appear in the knowledge graph wherever a plan mentions them.
- **Time span:** daily weather from 1 January 2000 to 31 December 2025, and crop statistics from 2000–01 to 2022–23.

1.6 The guardrail

Large language models are fluent but can state things that are not true, a problem known as hallucination [3]. In an advisory system for farmers this is unacceptable. The project therefore follows one non-negotiable rule: **the language model only rephrases facts retrieved from the knowledge graph and numbers already produced by the forecasting and yield models. It never predicts a yield or invents a recommendation, and every recommendation it gives cites the contingency-plan page it came from.**

1.7 Status at the end of the first phase

The project is planned over two semesters. The system is organised as a pipeline of ten stages grouped into four phases (Chapter 4). In the first phase the team completed the data audit and preprocessing, exploratory analysis, the weather-forecasting stage with a hurdle rainfall model as the main model and four comparative models, the knowledge graph with its query interface, and an MLOps pipeline around the forecasting models. The remaining stages, from season aggregation through yield prediction and human impact to the LLM layer, are planned for the second phase and are described in Chapter 8.

1.8 Organisation of the thesis

- **Chapter 2** reviews earlier work on rainfall forecasting, crop-yield prediction, drought indices and knowledge-graph-based question answering, and identifies the research gap.

- **Chapter 3** describes the study area and the four data sources, and the problems found while auditing and cleaning them.
- **Chapter 4** presents the overall architecture: the ten stages, how data flows between them, and which stages are complete.
- **Chapter 5** covers weather forecasting: the main two-stage hurdle model for rainfall, its results, and a comparative study with LSTM, SARIMA, XGBoost and TFT.
- **Chapter 6** explains how the contingency plans were turned into a knowledge graph and how the graph is queried.
- **Chapter 7** describes the MLOps pipeline that keeps the forecasting model up to date and monitored.
- **Chapter 8** concludes the first phase and lays out the detailed plan for the second.
- **Appendix A** lists model configurations, the graph schema and the interfaces.

Chapter 2

Literature Review

2.1 Introduction

The system in this thesis draws on four bodies of work: forecasting daily weather (especially rainfall), predicting crop yield with machine learning, defining and measuring drought, and grounding language-model answers in structured knowledge. This chapter reviews each in turn, keeping the focus on what matters for a small-data, district-level decision-support system, and ends by stating the gap the project addresses.

2.2 Weather and rainfall forecasting

2.2.1 Statistical time-series models

The classical approach to forecasting a weather series is the Box–Jenkins family of autoregressive integrated moving-average models and their seasonal extension, SARIMA [4]. They model the next value as a linear combination of past values and past errors, with seasonal terms for yearly cycles. Mulla et al. [5] applied seasonal ARIMA with exogenous variables (SARIMAX) to monthly rainfall in Phaltan taluka, Maharashtra, and compared model orders directly on R^2 and RMSE; their better configuration reached an R^2 of 0.91 on monthly totals. Monthly totals are, however, much smoother than daily rainfall, and the study covers a single rain gauge, so it does not show whether one model order carries across districts with different rainfall regimes. SARIMA's linear structure also struggles with the abrupt on/off behaviour of daily rain.

2.2.2 Gradient-boosted trees

Gradient-boosted decision trees such as XGBoost [6] build an ensemble of shallow trees, each correcting the errors of the previous ones. They handle non-linear interactions and mixed feature types well and need little tuning, which makes them strong baselines on tabular data built from lagged and rolling-window weather features. Dehvari et al. [7] used XGBoost on zenith wet delay series derived from GNSS receivers to predict rainfall events one hour ahead

across several stations. The short horizon and the need for GNSS-derived inputs limit how far the approach carries over to daily, district-level forecasting.

2.2.3 Recurrent neural networks

Long Short-Term Memory (LSTM) networks [8] are recurrent neural networks with gates that decide what to remember and what to forget, which lets them learn dependencies over many time steps. Bukhari and Ogudo [9] compared simple RNNs and LSTMs in single- and multi-variable settings on 24 years of daily India Meteorological Department data from three stations in northern India, forecasting temperature and rainfall. Their models forecast one day ahead from three input variables, with no added features or attention. In the present project an LSTM of this kind, extended to 11 variables, a 45-day lookback with engineered rolling features and a direct three-day horizon, is used as one of the comparative models.

2.2.4 Attention-based models

The Temporal Fusion Transformer (TFT) [10] combines recurrent layers with multi-head attention and variable-selection networks to produce multi-horizon forecasts whose inputs and time steps can be inspected. It performs well on large multivariate datasets, but it has many parameters and needs substantial data and tuning, which is a concern when each district has only about 9,500 daily rows.

2.2.5 Rainfall as a zero-inflated variable

Daily rainfall in a semi-arid region is zero on the large majority of days, and highly skewed on the days it does rain. Models that predict it as one continuous quantity tend to spread small amounts of rain across dry days and to under-predict heavy events. Statisticians have long modelled such variables in two parts: first whether the event happens, then how large it is given that it happens. This is the idea behind the *hurdle* and two-part models of econometrics [11,12], and behind classical daily-rainfall models that pair a wet/dry occurrence process with a separate amount distribution [13]. The main forecasting model of this project follows the same structure, with gradient-boosted trees for both parts.

2.3 Crop-yield prediction with machine learning

Van Klompenburg et al. [14] reviewed machine-learning studies of crop-yield prediction and found that temperature, rainfall and soil type are the most used features, and that random forests, gradient boosting, neural networks and linear regression are the most used models. Random forests [15] are popular because they cope with small, noisy datasets and give feature-importance estimates. Deep networks generally need far more samples than are available at district level. In this project each district-crop-season combination has only about 23 years of

yield records, which rules out deep learning for the yield stage and favours tree ensembles with a linear baseline.

A prediction alone does not help a district officer unless the reasons behind it are visible. SHAP (SHapley Additive exPlanations) [16] attributes each prediction to its input features using Shapley values from cooperative game theory, so that the contributions add up exactly to the difference between the prediction and the average prediction. It gives explanations of the form "the 21-day dry spell in August lowered the predicted soybean yield by 0.18 t/ha", which is the kind of statement the system aims to produce.

2.4 Drought definition and indices

Drought is usually divided into meteorological (lack of rain), agricultural (lack of soil moisture for crops), hydrological and socio-economic drought [17]. The most widely used meteorological index is the Standardised Precipitation Index (SPI) [18], which fits a probability distribution to a location's long-term rainfall record and expresses the current period's rainfall as a number of standard deviations from that location's own normal. This is what makes SPI suitable here: it judges each district against its own history instead of a fixed threshold. The contingency plans themselves use agronomic definitions instead: a monsoon onset delayed by two to eight weeks, two consecutive rainless weeks during crop growth, or early withdrawal of the monsoon. The project uses both: the plans' definitions to label past seasons and to look up actions, and SPI/tercile thresholds in the planned condition-classification stage.

2.5 Knowledge graphs and retrieval-augmented generation

A knowledge graph represents facts as nodes (entities) connected by typed edges (relations) [19]. Compared with a flat table, a graph answers multi-hop questions naturally, for example *"which actions does the Beed plan recommend for crops grown on the soil types that also occur in Latur?"* Graphs are also easy to inspect, and every fact can keep a pointer to its source.

Retrieval-augmented generation (RAG) [20] reduces hallucination by retrieving relevant documents first and asking the language model to answer from them. GraphRAG approaches retrieve sub-graphs or graph-derived summaries instead of plain text chunks [21], and the wider literature on unifying LLMs and knowledge graphs [22] argues that graphs supply the factual grounding and traceability that LLMs lack. For an advisory system this traceability matters as much as accuracy: a recommendation should be checkable against the official document.

2.6 Research gap

Table 2.1 summarises the forecasting studies that most directly informed the model choices. Taken together with the literature on yield prediction, drought indices and knowledge-graph question answering, the review shows the following gap.

1. Forecasting studies usually stop at the forecast. They do not carry it through to crop yield, human impact or recommended action.
2. Drought is often judged with a fixed threshold instead of against each district's own history and against the definitions the district's own plan uses.
3. The official contingency plans, which already encode expert agronomic knowledge for each district, have not been turned into a machine-readable form that a decision-support system can query.
4. LLM-based agricultural advisors rarely guarantee that every recommendation can be traced to an official source.

This project addresses all four by linking district-specific forecasting, history-relative condition classification, explainable yield prediction, Census-based impact estimation and a knowledge graph built from the contingency plans, with an LLM restricted to explaining retrieved facts.

Table 2.1: Forecasting studies that informed the model choices

Study	**Key contribution**	**Limitation for this project**
Dehvari et al. 2025 [7]	XGBoost on GNSS zenith wet delay predicts rainfall events one hour ahead across several stations.	One-hour horizon; needs GNSS data; evaluated mainly in the eastern USA.
Bukhari & Ogudo 2025 [9]	RNN vs. LSTM, single- and multi-variable, on 24 years of daily IMD data from three north-Indian stations.	One day ahead only; three input variables; no attention or engineered features.
Lim et al. 2021 [10]	Temporal Fusion Transformer: attention-based, interpretable multi-horizon forecasting.	Complex; high computational cost; needs substantial data and tuning.
Mulla et al. 2024 [5]	SARIMAX for monthly rainfall in Phaltan taluka, Maharashtra; R^2 0.91 for the better order.	Single station and monthly totals; does not test whether one order generalises across districts.

Chapter 3

Study Area and Data

3.1 Study area

The five districts were chosen because they share three properties that the system depends on.

1. **Recurring drought exposure.** Beed, Latur and Osmanabad lie on the Marathwada plateau; Ahmednagar and Solapur lie in the semi-arid rain-shadow belt of western Maharashtra. All five belong to the State's traditional scarcity zone, with a long record of failed or delayed monsoons.

2. **Consistent multi-source data.** Each district has 26 years of daily weather (2000–2025), 23 years of crop statistics (2000–01 to 2022–23) and Census 2011 demographics, which gives a uniform base for modelling.

3. **Official contingency plans exist.** An ICAR-format District Agriculture Contingency Plan is available for each of the five, and these plans are the knowledge base the recommendation layer is built on.

Table 3.1 summarises the climate of the five districts as found during exploratory analysis, and Figure 3.1 shows how much the Kharif-season rainfall varies from year to year.

Table 3.1: Rainfall characteristics of the five districts, 2000–2025 (NASA POWER daily data)

District	**Mean annual rain (mm)**	**Jun–Sep share**	**Longest dry spell (days/yr, avg.)**
Ahmednagar	677	80.1%	75.7
Beed	721	80.5%	72.2
Latur	762	78.9%	70.4
Osmanabad	752	76.8%	71.0
Solapur	622	73.8%	76.3

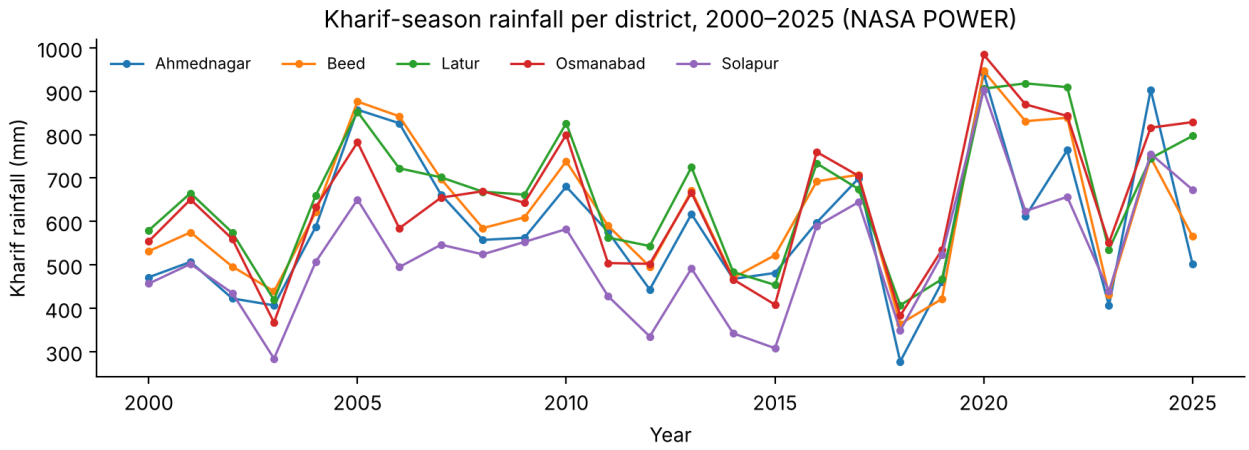


Figure 3.1: Kharif-season rainfall in each district, 2000–2025. Dips such as 2014 and 2018 correspond to known drought years.

The exploratory analysis brought out several facts that shaped the design:

- **High year-to-year variability.** Annual rainfall varies by 150 to 185 mm (about 20 to 25%) around each district's mean. A drought must therefore be judged against the district's own history, not a fixed threshold.
- **Strong monsoon dependence.** Between 74 and 81% of the annual rainfall falls in June to September. The other eight months are structurally dry, which supports aggregating weather to the season level for the yield model.
- **Extreme dry spells.** The longest recorded dry spell reaches 125 to 133 days in Solapur and Beed, more than four months without meaningful rain.
- **Rainfall is trending upward.** Comparing the first and the latest decade, annual rainfall rose by 2% (Latur) to 16% (Solapur), so models trained on older years may under-predict recent rainfall.
- **Temperature is fairly uniform** across districts: daily means of 25.7 to 26.8 °C, summer highs near 44 to 45 °C and winter lows of 7 to 9 °C.

3.2 Data sources

The project uses four public data sources, summarised in Table 3.2.

Table 3.2: Data sources used in the project

Source	Coverage	Content	Used for
NASA POWER [23] daily weather	1 Jan 2000 – 31 Dec 2025, one file per district	11 daily variables: rainfall, mean/max/min temperature, relative humidity, wind speed at 2 m and 50 m, surface pressure, solar radiation, dew point, cloud cover	Weather forecasting, season summaries, condition labelling
DES district crop statistics [24]	2000–01 to 2022–23	Area, production and yield per district, crop and season	Yield-model target; which crops each district grows
ICAR-CRIDA District Agriculture Contingency Plans [2]	5 PDFs, 2011	District profile (rainfall normal, soils, land use, irrigation) and condition→action tables	Knowledge graph
Census of India 2011 [25]	District level	Population, density, sex ratio, decadal growth; total workers 1991/2001/2011	Human-impact layer only

3.2.1 Weather data

The weather files come from NASA's POWER project, which provides daily values derived from satellite observations and reanalysis on a regular grid. Each district file has one row per day from 2000 to 2025 (about 9,500 rows) and, after cleaning, no missing dates. Because POWER values are grid averages, they smooth out local extremes: a cloudburst over one village is spread over the whole cell. This matters later when heavy-rain events are labelled (Chapter 6).

3.2.2 Crop statistics

District-wise area, production and yield come from the Directorate of Economics and Statistics (DES), Ministry of Agriculture and Farmers Welfare. Table 3.3 summarises the crop-season combinations available.

Table 3.3: Crop-season combinations in the DES data (five districts, 2000–01 to 2022–23)

Crop (season)	Mean area (ha)	Mean yield (t/ha)	Missing
Arhar/Tur (Kharif)	54,261	0.61	0%
Bajra (Kharif)	71,562	0.53	0.9%
Jowar (Kharif)	40,904	0.78	7.0%
Jowar (Rabi)	305,949	0.64	0%
Soybean (Kharif)	109,932	1.07	0%
Sunflower (Kharif)	8,336	0.46	0.9%
Sunflower (Rabi)	14,320	0.55	6.1%

Rabi Jowar is by far the largest crop by area (about 306,000 ha on average), and Soybean has the highest mean yield. Yields differ sharply between districts: Kharif Jowar averages 1.10 t/ha in Latur but only 0.56 t/ha in Solapur. Each district-crop-season has at most 23 yearly records, which is why deep learning is not used for yield.

3.2.3 Contingency plans

The five plans were prepared in 2010–2011 in a common ICAR template. Pages 1–2 contain the district profile: rainfall normal, soil types with their area, land use and irrigated area. Section 2, "Strategies for weather related contingencies", contains the condition→action tables that form the heart of the knowledge graph (Chapter 6). Table 3.4 shows the profile values extracted from them.

Table 3.4: District profile extracted from the contingency plans

District	Rainfall normal (mm)	Net irrigated area	Dominant soil (share of area)
Ahmednagar	419.0	not stated	Shallow grey soils (389 kha)
Beed	605.4	16.0%	Shallow black soils (58.9%)
Latur	634.9	not captured	Shallow soils (49.4%)
Osmanabad	693.9	15.6%	Shallow soils (66.1%)
Solapur	550.5	not stated	Shallow black soil (67.8%)

3.2.4 Census data

Census 2011 tables give each district's population, density, sex ratio and decadal growth, and the total number of workers in 1991, 2001 and 2011. These are used only *after* a yield has been predicted, to estimate how many people are exposed to a loss. They are never used as yield-model features, because there is no agronomic reason why population should change yield, and adding such features to a model trained on about 23 points per crop would invite spurious fits.

3.3 Data audit and preprocessing

Before any modelling, every file was opened and checked. The main problems found, and how they were handled, are listed below.

1. **Mislabelled files.** The DES and Census ".xls" files are actually HTML pages saved with an Excel extension. They are read with `pandas.read_html` and saved as clean CSV files.

2. **Inconsistent district names.** The same district is spelt differently in different sources (Table 3.5). A single mapping module (`district_name_map.py`) translates every spelling to one canonical name.

3. **Inconsistent crop names.** DES says "Bajra" while the plans say "Pearl millet" or "Pearlmillet"; "Tur", "Arhar" and "Pigeon pea" are the same crop. The same module maps crop names and their aliases.

4. **Agricultural years.** DES reports years as "2000–2001". These are converted to an integer crop year for joining with weather.

5. **Genuine gaps.** Some DES cells are empty (for example Ahmednagar Kharif Jowar from 2009–10 to 2015–16). These are reporting gaps, not zero yields, and are kept as missing.

6. **Missing crop.** Rabi Gram (chickpea), an important crop in all five districts, is absent from the downloaded DES table and must be sourced separately.

7. **Weather gaps.** The few gaps in the weather files are filled in the preprocessing notebook, which writes a clean copy used by all models. Rainfall and wind speed, which are highly skewed, are log-transformed before training.

8. **Unrelated material.** One review document found in the shared folder belonged to a different project scope and was excluded.

Table 3.5: District name variants across sources

Canonical	DES	Census 2011	Weather / plans
Ahmednagar	Ahilyanagar	Ahmadnagar	Ahmednagar
Beed	Beed	Bid	Beed
Osmanabad	Dharashiv	Osmanabad	Osmanabad
Latur	Latur	Latur	Latur
Solapur	Solapur	Solapur	Solapur

3.4 Summary

The four sources together give 26 years of daily weather, 23 years of crop yields, the official expert guidance and the population base for every district. Their main weaknesses are known and recorded: daily rainfall is very skewed and hard to forecast, the crop records are short, Rabi Gram is missing, and the contingency plans and Census figures are single snapshots from 2011.

Chapter 4

System Architecture and Methodology

4.1 Overview

The system is a pipeline of ten stages grouped into four phases (Figure 4.1). A user chooses a district and a season. The weather for the coming days is forecast, the season is summarised and classified, crop yield is predicted and explained, the loss is turned into human impact, and finally the matching official guidance is retrieved and explained in plain language. Each stage writes a well-defined output that the next stage reads, so stages can be built, tested and replaced independently.

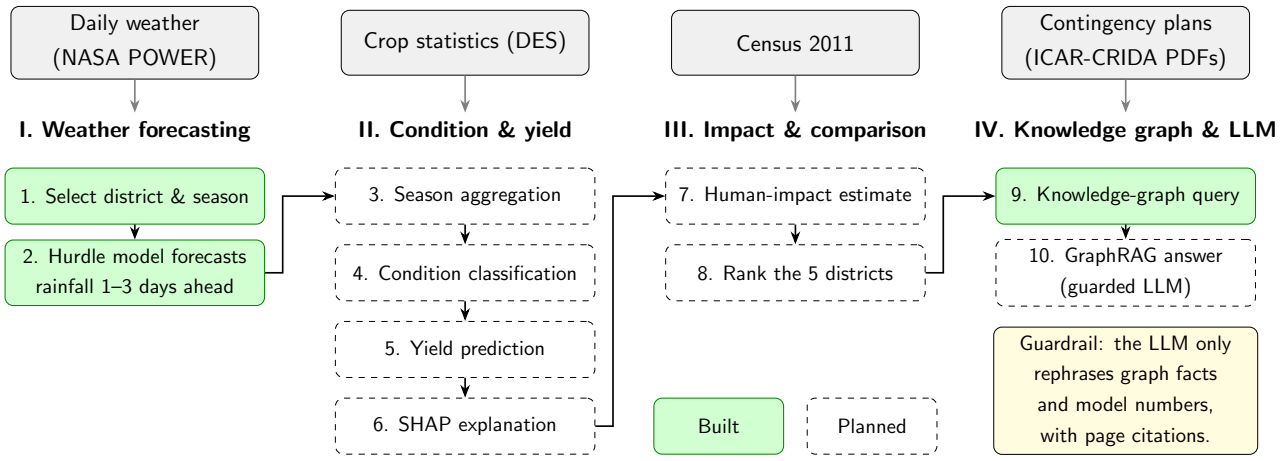


Figure 4.1: System architecture: ten stages in four phases. Green boxes are built; dashed boxes are planned for the second phase.

4.2 The ten stages

Table 4.1 describes what each stage does in plain terms and its status at the end of the first phase.

Table 4.1: Pipeline stages and their status

#	Stage	What it does	Status
1	Selection	The user chooses a district and a crop-season (Kharif or Rabi).	Done
2	Weather forecast	A two-stage hurdle model trained on that district's history forecasts daily rainfall 1 to 3 days ahead (LSTM, SARIMA, XGBoost and TFT were evaluated as comparative models).	Done
3	Season aggregation	Daily weather is summarised into season rainfall, longest dry spell, monsoon onset date, etc.	Planned
4	Condition classification	The season is labelled normal, dry spell, deficit or drought-like against the district's own history (SPI/terciles and the plans' definitions).	Planned[†]
5	Yield prediction	Random forest / gradient boosting (with a linear baseline) predict yield per district, crop and season.	Planned
6	Explanation	SHAP shows which weather factors pushed each predicted yield up or down.	Planned
7	Human impact	The yield shortfall is converted into people and workers affected using Census data.	Planned
8	District comparison	Severity and exposure scores are normalised and combined to rank the five districts.	Planned
9	Knowledge-graph query	The graph returns the official actions for the district, crop, season and condition, with page citations.	Done
10	GraphRAG answer	An LLM extracts entities from a question, retrieves the subgraph and writes one grounded answer.	Planned

[†] A rule-based season labeller built for the knowledge graph already labels all past seasons and acts as a stand-in for Stage 4 (Section 6.4).

4.3 Key design decisions

1. **One model per district.** The five districts have different rainfall regimes (Table 3.1). Training a separate forecaster for each lets every model learn its own district's seasonality and avoids one district's patterns dominating another's.

2. **Self-trained forecasting.** The forecast is produced from the district's own historical record, so the system keeps working without depending on an external live weather service.

3. **A model built for rainfall.** Daily rainfall is zero on most days and highly skewed when it rains, so the main forecaster is a two-stage hurdle model that predicts rain occurrence and amount separately. Deep learning (LSTM, TFT) was evaluated in a comparative

study; for the yield stage, with only about 23 seasons per district-crop, tree ensembles and a linear baseline are used.

4. **History-relative drought.** Conditions are judged against each district's own 2000–2025 record and the plans' own definitions, never against one national threshold.

5. **Census only after prediction.** Population and workforce enter only the impact stage, never the yield model.

6. **Structured knowledge, not free text.** The contingency plans are turned into a graph whose every edge records the source page, so any recommendation can be traced.

7. **A guarded LLM.** The LLM sees only retrieved graph facts and model outputs, and its job is to phrase them clearly.

8. **Replaceable stages with fixed interfaces.** For example, the graph reads season labels from a CSV with fixed columns; when Stage 4 is built it simply writes a file with the same columns, and the graph picks it up with no change to its schema.

4.4 How a query flows through the system

The following example shows how the finished system is intended to answer a question; Stages 2 and 9 already work today.

1. **Question:** "Rain has stopped in Beed for two weeks while the soybean is flowering. What should be done?"

2. **Stage 10** extracts the entities: district = Beed, crop = Soybean, season = Kharif, condition = mid-season dry spell at flowering.

3. **Stages 2–4** confirm the condition from the forecast and the season-to-date weather.

4. **Stages 5–8** predict the soybean yield for the season, explain the drivers, estimate the people affected and place Beed in the district ranking.

5. **Stage 9** retrieves the Beed plan's recommendations for this situation, for example "give protective irrigation with sprinkler" (Beed plan, p. 18), together with past seasons in which Beed had the same condition and the yield that followed.

6. **Stage 10** writes one answer from these pieces only, citing the plan pages.

4.5 Technology stack

- **Language and data:** Python, pandas, NumPy; Jupyter notebooks for exploration.

- **Forecasting:** scikit-learn [26] for the main hurdle model; for the comparative study, PyTorch [27] (LSTM, TFT), XGBoost and statsmodels (SARIMA).
- **Knowledge graph:** pdfplumber for table extraction, NetworkX [28] for the graph, GraphML/JSON export, Neo4j import scripts, vis-network for the interactive viewer.
- **MLOps:** Git and GitHub Actions, DVC with a DagsHub remote, MLflow [29] model registry, FastAPI service, Docker, pytest.
- **Planned:** SHAP for explanations; LangChain/LlamaIndex for GraphRAG; Streamlit for the user interface.

Chapter 5

Weather Forecasting

5.1 What this stage has to do

Stage 2 takes the recent daily weather of a district and forecasts the coming days. Its output feeds the season summary (Stage 3): to judge whether the current season is heading into a dry spell or a delayed onset, the system needs both the weather observed so far and a view of the days ahead. The forecasting model has to work without calling an external weather service, so it is trained only on the district's own 26-year record.

Of all the weather variables, **rainfall** matters most here. Every condition in the contingency plans (delayed onset, dry spell, terminal drought, excess rain) is defined by rainfall, and rainfall is the main weather driver of rainfed crop yield. The project therefore uses a model designed specifically for daily rainfall, the **two-stage hurdle model**, as its **main forecasting model**, used one per district throughout the pipeline. Four other well-known approaches, an LSTM network, SARIMA, XGBoost and a Temporal Fusion Transformer, were trained on the same data as a **comparative study**, to check that the choice of the hurdle model is justified.

One rule applies to every model in this chapter: a forecast may use only information that would really be available on the day the forecast is made.

5.2 Why daily rainfall needs a special model

Daily rainfall behaves very differently from smooth variables such as temperature (Figure 5.1):

- **Most days are dry.** In every district about 62–64% of days have no measurable rain (0.25 mm or less), and 80–83% have less than 2.5 mm, the India Meteorological Department's threshold for a "rainy day".
- **The rain that does fall is highly skewed.** On rainy days the median amount is only about 2–3 mm, yet the heaviest 10% of rainy days deliver about 42–44% of all the rain.

A model that predicts rainfall as one continuous number, and is trained to keep its average error small, learns a poor compromise: it predicts "a little rain" on almost every day. That

is never badly wrong on dry days, but it blurs the difference between dry and wet days and badly under-predicts heavy rain. Both are exactly what a drought system needs to get right.

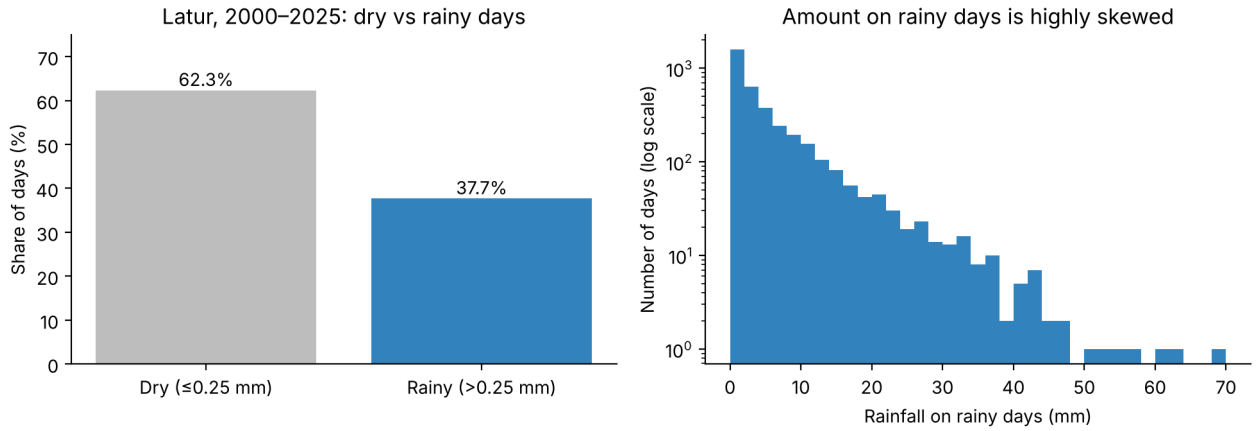


Figure 5.1: Daily rainfall in Latur, 2000–2025. Left: about 62% of days are dry. Right: rainy-day amounts are heavily skewed (note the log scale). The other four districts look the same.

Statisticians have long modelled such variables in two parts: first *whether* the event happens, then *how large* it is given that it happens [11, 12]. Classical daily-rainfall models do the same, pairing a wet/dry occurrence process with a separate distribution for the amount [13]. The hurdle model follows this idea.

5.3 Evaluation set-up

5.3.1 Chronological split

Weather is a time series, so data are never shuffled; every model is trained on the past and tested on a later period it has never seen. For the hurdle model the split is made on the date being forecast (the *target* date):

- **Test:** target dates from 1 January 2022 onwards (about 1,718 days per horizon); all results below are on this period;
- **Validation:** the 365 days before the test period;
- **Training:** everything earlier, from 2000.

5.3.2 Metrics

For n test days with observed values y_t, forecasts $\hat{y}_t$ and observed mean $\bar{y}$:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}, \qquad \mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|, \tag{5.1}$$

$$R^2 = 1 - \frac{\sum_t (y_t - \hat{y}_t)^2}{\sum_t (y_t - \bar{y})^2}. \tag{5.2}$$

RMSE and MAE are in millimetres; RMSE punishes large misses more heavily. R^2 is unit-free: 1 means perfect, 0 means no better than always predicting the test-period average, and negative values mean worse than that. For daily rainfall a few heavy days dominate the sum of squares, so even a useful model has a modest R^2.

5.4 The hurdle model (main model)

5.4.1 Structure

For each district and each forecast horizon $h \in \{1, 2, 3\}$ days, the model has two parts (Figure 5.2):

1. **Hurdle 1, the rain gate.** A gradient-boosted classifier estimates the probability p that day $t + h$ will have more than 0.25 mm of rain.

2. **Hurdle 2, the rain depth.** A gradient-boosted regressor, trained only on rainy days, predicts $\log(1 + \text{rainfall})$ on day $t + h$. Working on the log scale tames the skew of heavy events.

The forecast combines the two:

$$\hat{y}_{t+h} = \begin{cases} \exp\big(g_h(\mathbf{x}_t)\big) - 1 & \text{if } p_h(\mathbf{x}_t) > \tau, \\ 0 & \text{otherwise,} \end{cases} \qquad \tau = 0.35, \tag{5.3}$$

where $\mathbf{x}_t$ are the features available on forecast day t, p_h is the gate and g_h the depth model. The threshold $\tau = 0.35$ is set below 0.5 on purpose: because rainy days are the minority, a gate that waited for 50% certainty would miss too many of them. Training separate models for each horizon (*direct* multi-step forecasting) avoids feeding one day's forecast into the next, which would compound errors.

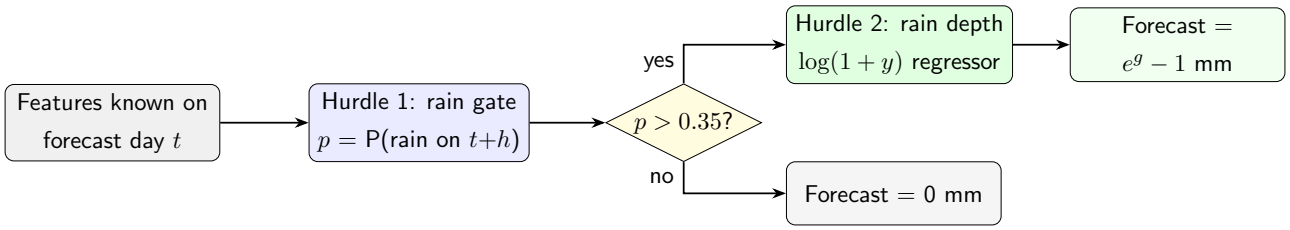


Figure 5.2: The two-stage hurdle model. One gate and one depth model are trained for each district and each horizon $h = 1, 2, 3$ days.

5.4.2 Features

Every feature is computed only from data available on the forecast day t; rolling rainfall statistics end on the day *before* t, so they never include the rain being predicted. The 21 features are:

- **Season:** $\sin(2\pi d/365.25)$ and $\cos(2\pi d/365.25)$ of the day of year d, which tell the model where it is in the monsoon cycle and place 31 December next to 1 January;
- **Recent rain:** rainfall 1, 2 and 3 days earlier; mean rainfall over the previous 3, 7 and 14 days; maximum rainfall in the previous 3 days;
- **Moisture and pressure tendency:** humidity and pressure on the previous day and their day-to-day change; a falling pressure and rising humidity often precede rain;
- **Current conditions:** relative humidity, surface pressure, mean, maximum and minimum temperature, wind speed, solar radiation and dew point.

5.4.3 Training settings

Both parts use scikit-learn's histogram-based gradient boosting [26]: the gate with 250 boosting iterations and a learning rate of 0.04, the depth model with 350 iterations and a learning rate of 0.03, both with L2 regularisation of 0.1 and a fixed random seed. All settings live in one configuration file (Appendix A). Training needs no GPU and runs quickly on an ordinary laptop CPU, which makes regular retraining practical (Chapter 7).

5.4.4 Model development: from nowcast to forecast

The first version of the hurdle model also used physically motivated features (temperature range, dew-point spread, wind shear, vapour pressure deficit and a moisture-flux proxy) and a rule that set rainfall to zero when the air was clearly too dry. It scored very highly, with R^2 between 0.79 and 0.94. A careful review while building the MLOps pipeline showed why: its rolling rainfall features included the day being predicted, its humidity and temperature inputs were from that same day, and its target was same-day rainfall. In other words, it estimated today's rain from today's weather, a *nowcast*, and was partly told the answer. This is a classic case of *data leakage* [30]. Figure 5.3 shows the symptom: the predicted series traces the observed one almost perfectly and with no delay, which no genuine forecast of daily rain can do.

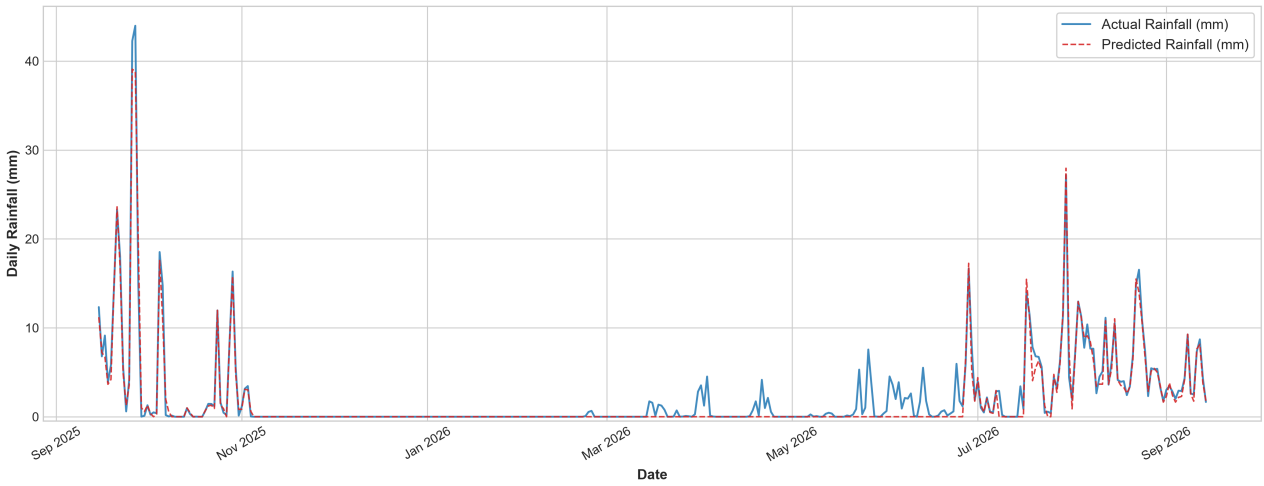


Figure 5.3: First (same-day) version of the hurdle model, Latur, one-year validation window ($R^2 = 0.94$, RMSE 1.17 mm, MAE 0.51 mm). Because its inputs included the same day's rainfall, it reproduces the observed series almost exactly; it is a nowcast, not a forecast. The corresponding values for the other districts were R^2 0.92 (Ahmednagar), 0.94 (Beed), 0.88 (Osmanabad) and 0.79 (Solapur).

The model was therefore rebuilt as a genuine forecaster: features shifted so they end before the forecast day, targets moved h days ahead, separate models per horizon, and the split made on the target date. All results in this chapter are for this corrected version, which is the one used in the project.

5.4.5 Results

Table 5.1 gives the test results for all five districts and all three horizons.

Table 5.1: Hurdle model: rainfall forecast test results (target dates from 2022 onward)

	Day +1			**Day +2**			**Day +3**		
District	R^2	RMSE	MAE	R^2	RMSE	MAE	R^2	RMSE	MAE
Ahmednagar	0.151	7.09	1.62	0.103	7.29	1.74	0.072	7.41	1.80
Beed	0.259	4.74	1.62	0.190	4.96	1.77	0.147	5.08	1.77
Latur	0.289	4.50	1.80	0.186	4.81	1.99	0.154	4.91	2.04
Osmanabad	0.261	4.72	1.89	0.173	4.99	2.04	0.134	5.10	2.10
Solapur	0.282	4.12	1.59	0.150	4.49	1.73	0.106	4.60	1.82

RMSE and MAE in mm/day.

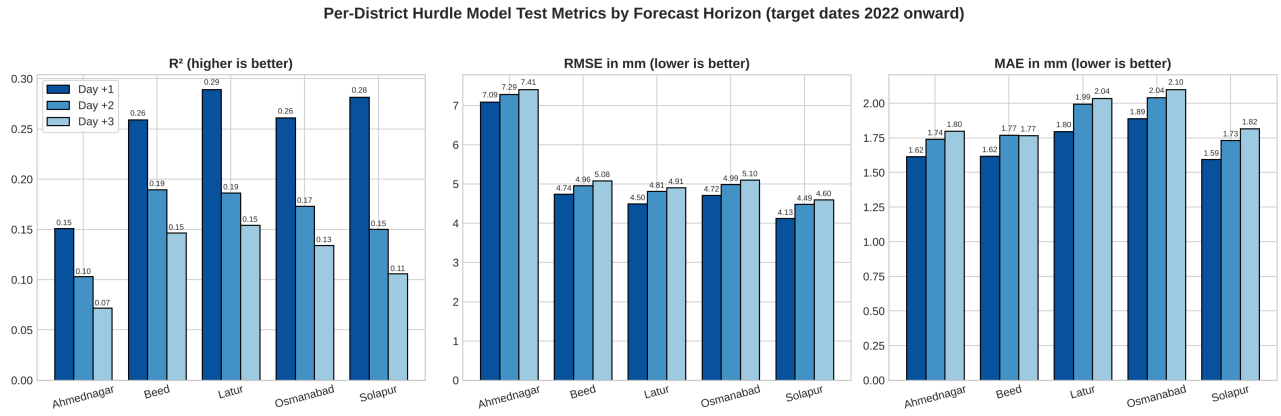


Figure 5.4: Hurdle model test metrics for each district and forecast horizon.

- **One day ahead**, the model explains 26–29% of the day-to-day variation in rainfall in four of the five districts, with a typical error (MAE) of 1.6–1.9 mm.
- **Skill falls with lead time**, to 0.07–0.15 by day 3. This is expected for point rainfall in a monsoon climate: whether it rains on a particular day a few days ahead is hard to predict from local history alone.
- **Ahmednagar is the hardest district.** Its RMSE is much larger because its record contains a few extreme events (one day exceeds 250 mm) that dominate the squared error.

How well the gate separates dry and rainy days. Because the drought rules in the plans depend on runs of dry days, the most important job of the model is to say correctly whether a day will be dry or rainy. Figure 5.5 evaluates Hurdle 1 on its own. One day ahead it classifies 84–88% of days correctly and detects 89–92% of rainy days; by day 3 accuracy is still 81–86% and 84–87% of rainy days are caught. About one in five of its "rain" calls turns out dry (18–26% false alarms), which is the price of the deliberately low threshold $\tau = 0.35$: the gate prefers to catch rainy days, because a missed rainy day would wrongly extend a dry-spell count.

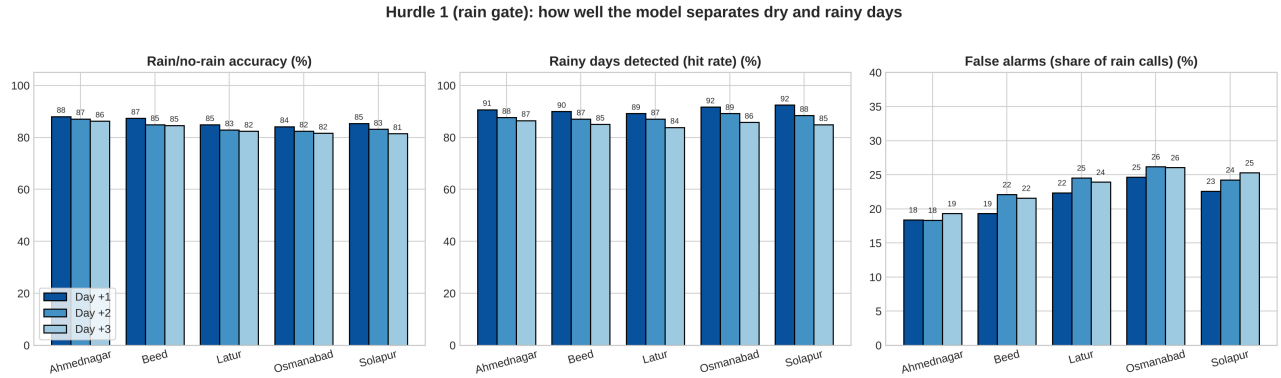


Figure 5.5: Rain gate (Hurdle 1) on the test period: overall accuracy, share of rainy days detected, and share of rain forecasts that turned out dry.

Forecasts over time. Figure 5.6 shows the one-day-ahead forecasts for all five districts over the last twelve months of the test period, and Figure 5.7 zooms into the 2025 Kharif season in Latur. The model correctly keeps forecasts at zero through the dry months, and during the monsoon it follows wet and dry spells, but it rounds off the peaks: a 40 mm day is typically forecast as 5–15 mm, and the three-day-ahead forecast is smoother still.

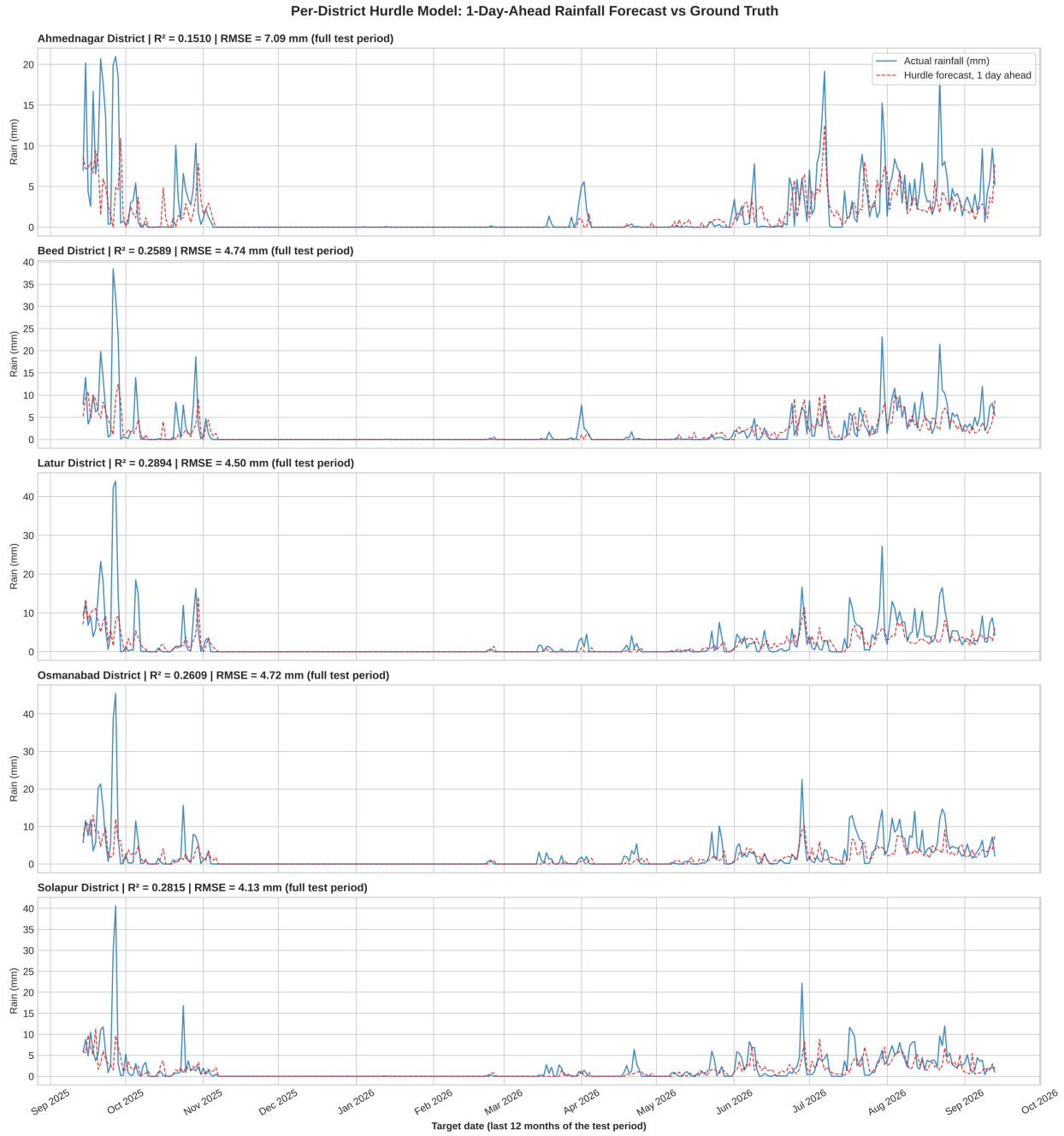


Figure 5.6: Hurdle model: one-day-ahead rainfall forecast against observed rainfall for each district, September 2025 to September 2026. R^2 and RMSE in the panel titles are for the full test period (2022 onward).

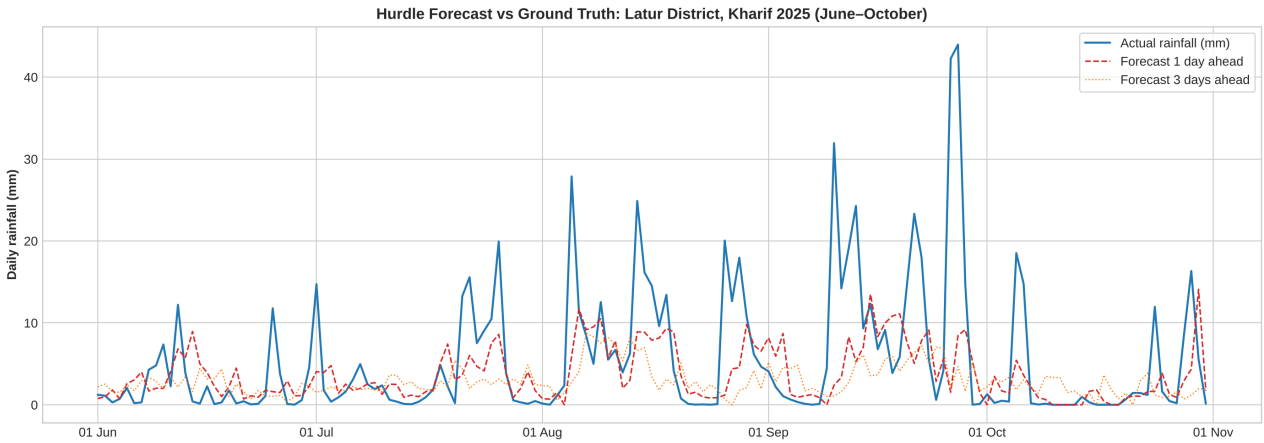


Figure 5.7: Hurdle model, Latur, Kharif 2025: forecasts one and three days ahead against observed rainfall.

What the model is good for in this system follows from these numbers. It gives a reliable short-range picture of whether the next one to three days are likely to stay dry, which is what decides whether a dry spell is continuing. Its gate also produces a *probability of rain* that can be shown to users directly. It is not a season-long forecast; Stages 3–4 will combine it with the rainfall already observed in the season (Chapter 8).

5.5 Comparative study

To test whether the hurdle model is the right choice, four other approaches were trained per district on the same weather data.

5.5.1 LSTM network

A Long Short-Term Memory network [8] reads a sequence one day at a time and keeps an internal memory, with gates that learn what to keep and what to forget. One model was trained per district.

Inputs. A window of the last 45 days, with 19 features per day: the 11 raw variables (rainfall, mean/maximum/minimum temperature, relative humidity, wind speed at 2 m and 50 m, surface pressure, solar radiation, dew point, cloud cover) and 8 rolling features (rainfall totals over 7, 14 and 30 days; 7-day means of humidity, solar radiation, wind, wind at 50 m and cloud cover), plus day-of-year sine/cosine. Rainfall and wind speed were log-transformed, and all values scaled using the training period only.

Architecture and training. One LSTM layer of 128 units, followed by dense layers of 384 and 192 units with dropout 0.25, producing all 11 variables for the next 3 days in one output (33 numbers). Training used the Huber loss [31], which behaves like squared error for

small mistakes and like absolute error for large ones,

$$\mathcal{L}_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \le \delta, \\ \delta\,|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise,} \end{cases} \tag{5.4}$$

with the Adam optimiser [32] (learning rate 5×10^{-4}, batch size 16) and early stopping after 12 epochs without validation improvement. The data were split into training (to 2018), validation (2019–2021) and test (2022 onward). Figure 5.8 shows the loss curves for Latur.

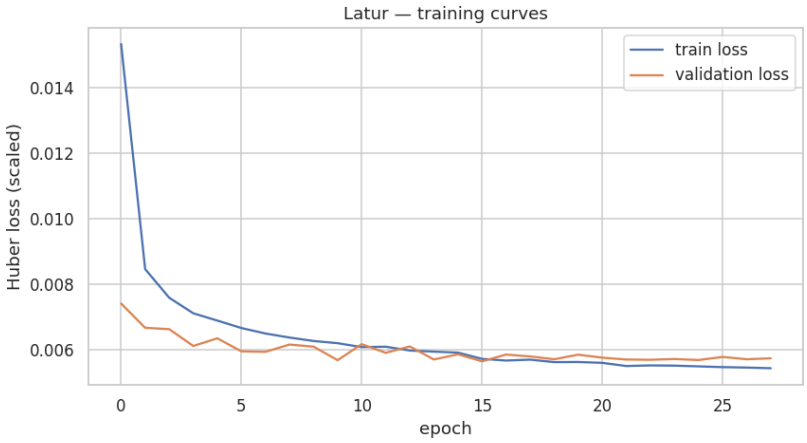


Figure 5.8: Comparative LSTM: training and validation loss for Latur.

Results. Because the LSTM forecasts all 11 variables, it also shows how forecastable the other variables are (Figure 5.9, Table 5.2). Smooth variables are forecast well: temperature, humidity and dew point reach R^2 of 0.85–0.94 in every district. Rainfall is by far its weakest variable (R^2 0.10–0.22): as Figures 5.10 and 5.11 show, its rainfall forecasts are squeezed towards small values and heavy rain is missed, exactly the behaviour described in Section 5.2.

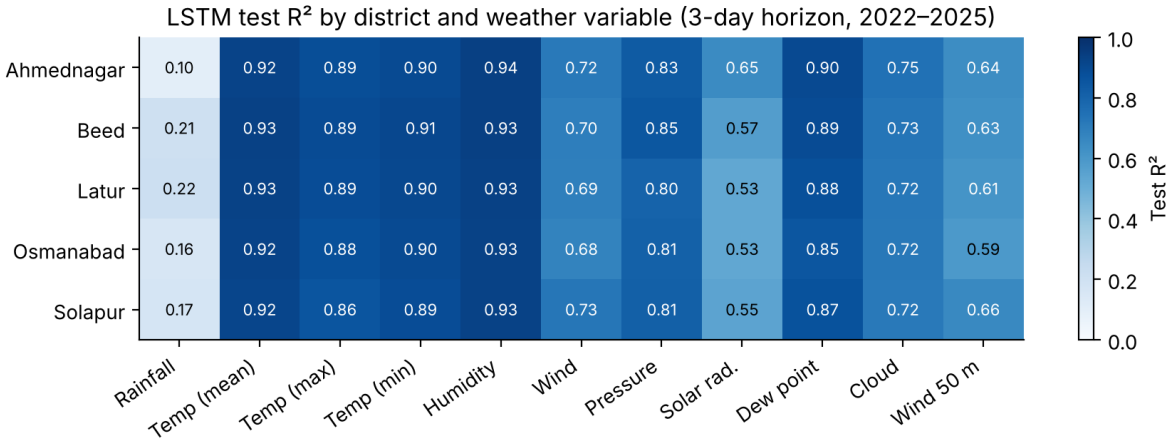


Figure 5.9: Comparative LSTM: test R^2 for every district and variable.

Table 5.2: Comparative LSTM: test results per district (2022 onward)

District	Average R^2 (11 variables)	Rainfall R^2	Rainfall MAE (mm)
Ahmednagar	0.749	0.101	1.67
Beed	0.750	0.209	1.68
Latur	0.738	0.218	1.92
Osmanabad	0.725	0.163	1.98
Solapur	0.736	0.169	1.68

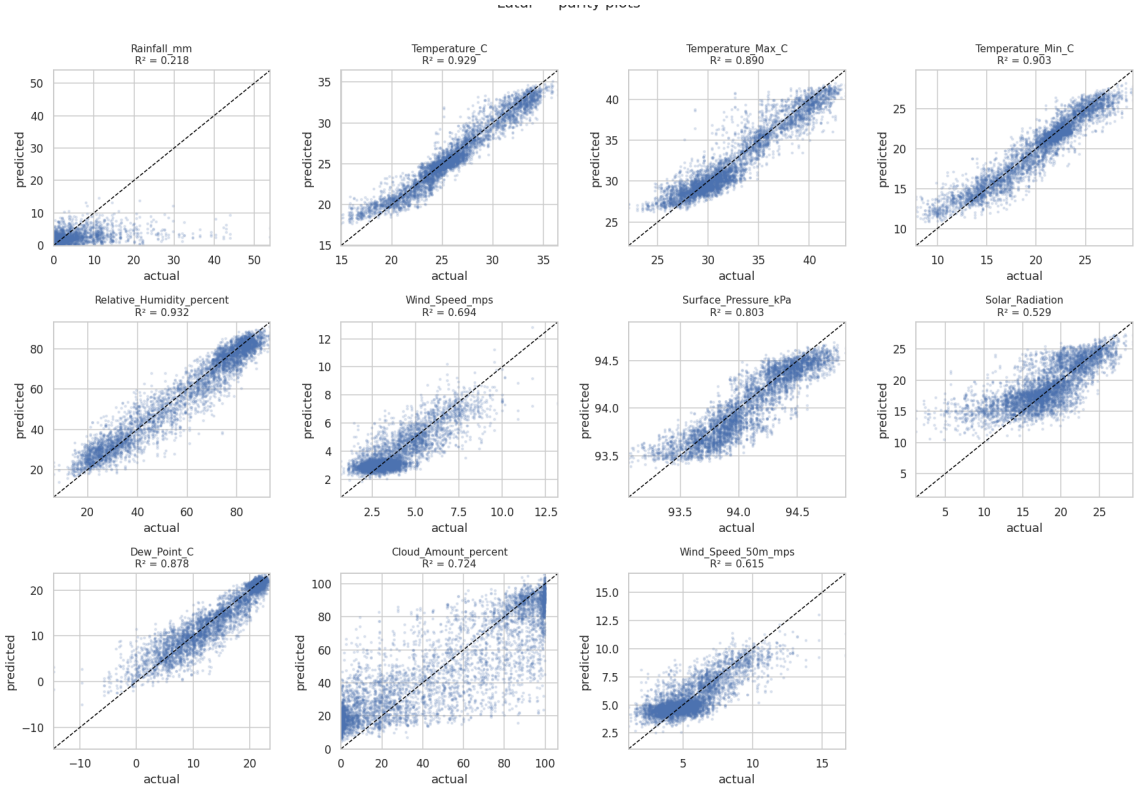


Figure 5.10: Comparative LSTM, Latur: predicted against observed values on the test set. Temperature and humidity lie close to the diagonal; rainfall forecasts are compressed towards small values.

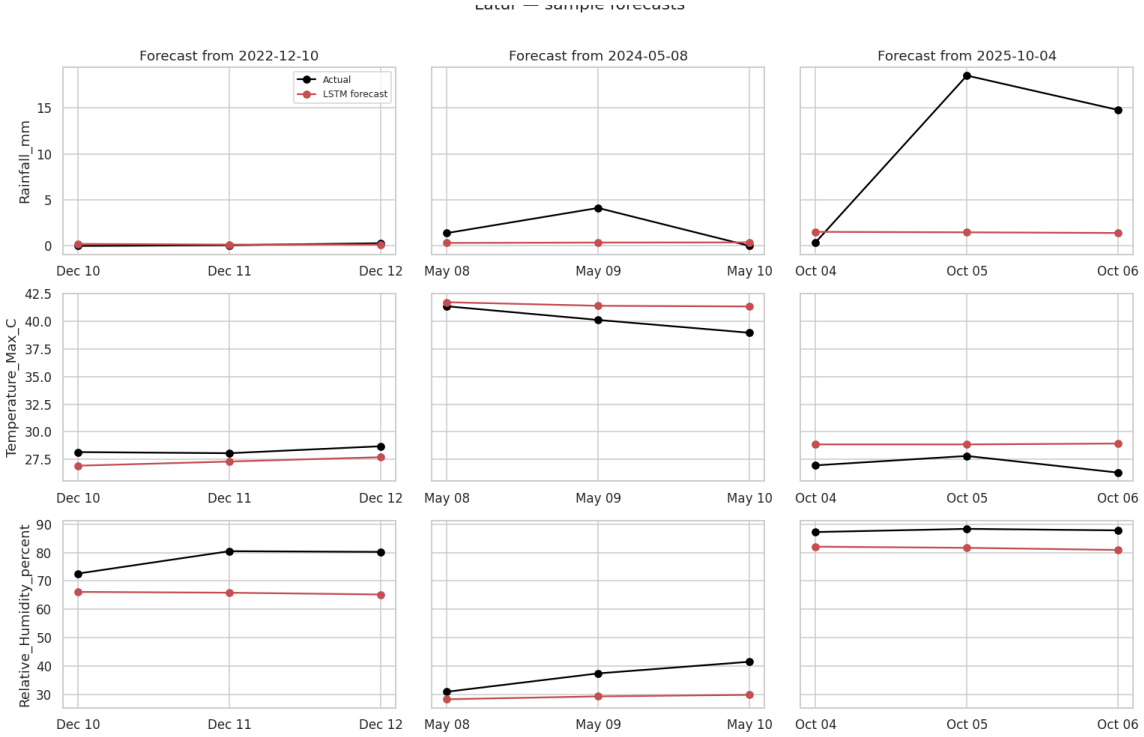


Figure 5.11: Comparative LSTM, Latur: three sample 3-day forecasts. Temperature and humidity follow the observed trend; the heavy rain of 5–6 October 2025 is missed.

5.5.2 SARIMA

A seasonal ARIMA model [4] was fitted per district as the statistical baseline. It forecasts the next value as a linear function of recent values, recent errors and the values one season earlier. It captures regular seasonality but assumes linear relations, so it struggles with the abrupt on/off behaviour of daily rain.

5.5.3 XGBoost

An XGBoost model [6] was trained per district on lagged and rolling-window weather features and seasonal terms. Gradient-boosted trees capture non-linear interactions, but used as a single regressor XGBoost treats rainfall as one continuous quantity and does not separate "will it rain?" from "how much?". The hurdle model uses the same family of learners but splits the problem in two.

5.5.4 Temporal Fusion Transformer

The TFT [10] was trained as a multivariate, multi-horizon forecaster with attention over past time steps and learnt input selection. It is the most complex model in the study, and with about 9,500 daily rows per district it has comparatively little data to learn from. Like the LSTM, it predicts rainfall as a continuous value and has no explicit treatment of dry days.

5.6 Comparison of results

5.6.1 Hurdle model and LSTM under the same protocol

The LSTM and the hurdle model were both evaluated as genuine forecasts on the same test period (2022 onward), so their rainfall results can be compared directly (Table 5.3, Figure 5.12).

Table 5.3: Rainfall test R^2: main hurdle model vs comparative LSTM

	Hurdle (main)				LSTM
District	**Day +1**	**Day +2**	**Day +3**	**Mean**	**Days +1..+3**
Ahmednagar	0.151	0.103	0.072	0.109	0.101
Beed	0.259	0.190	0.147	0.199	0.209
Latur	0.289	0.186	0.154	0.210	0.218
Osmanabad	0.261	0.173	0.134	0.189	0.163
Solapur	0.282	0.150	0.106	0.179	0.169

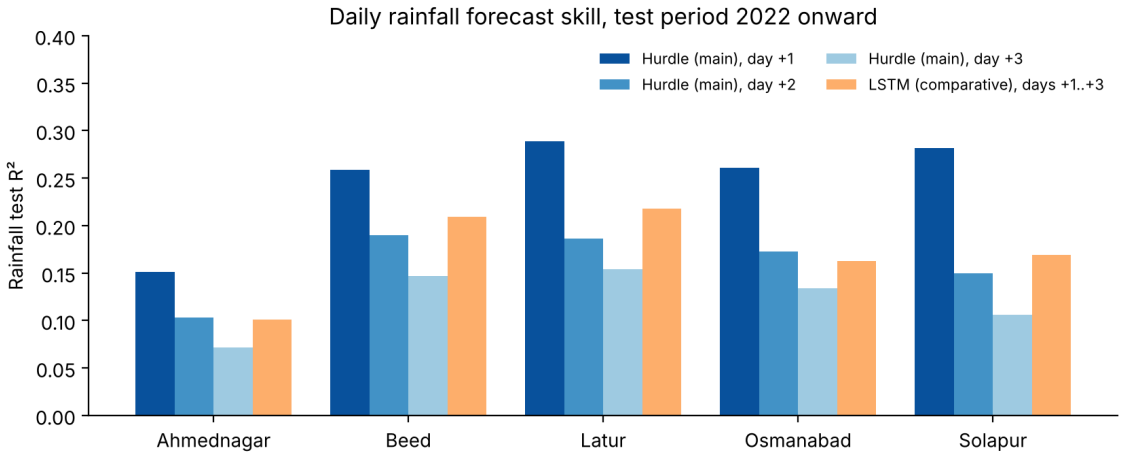


Figure 5.12: Daily rainfall forecast skill of the main hurdle model at each horizon, compared with the LSTM.

One day ahead, the hurdle model is better than the LSTM's rainfall forecast in every district. Averaged over the three days the two are close: the hurdle model is ahead in three districts and slightly behind in Beed and Latur.

5.6.2 All five models as presented at the first review

Table 5.4 reproduces the test R^2 values presented at the first review. These numbers come from each model's own experimental set-up and are **not directly comparable**: the LSTM column averages all 11 variables (most of which are easy), the hurdle column is from the first, same-day version described in Section 5.4.4, and the models differ in splits, targets and scaling.

They are shown for completeness. Re-running SARIMA, XGBoost and TFT under the same protocol as Table 5.3 is part of the second-phase plan.

Table 5.4: Test R^2 of the five models as presented at the first review (individual set-ups)

District	Hurdle[b]	SARIMA	XGBoost	LSTM[a]	TFT
Ahmednagar	0.923	0.432	0.303	0.749	0.511
Beed	0.940	0.479	0.506	0.750	0.439
Latur	0.945	0.592	0.590	0.738	0.500
Osmanabad	0.885	0.544	0.549	0.725	0.439
Solapur	0.787	0.512	0.509	0.736	0.372

[a] Average over all 11 variables. [b] First (same-day) version; superseded by Table 5.1.

5.7 Why the hurdle model is the main model

1. **It is built for rainfall.** It separates "will it rain?" from "how much?", which matches how daily rainfall actually behaves (Section 5.2). None of the comparative models does this.

2. **It gives the better next-day rainfall forecast** of the two models evaluated under the same protocol, in all five districts, and is on par with the LSTM over three days.

3. **Its output is directly useful.** Besides the amount, the gate gives a probability of rain and a forecast of exactly 0 mm on dry days, and it separates dry from rainy days correctly 81–88% of the time up to three days ahead. This is what the dry-spell and onset rules in Stages 3–4 need.

4. **It is light and easy to maintain.** It trains quickly on an ordinary CPU without a GPU, and is simple to version, retrain and monitor; it is the model run in the MLOps pipeline (Chapter 7).

5. **It is easier to explain.** Gradient-boosted trees work with SHAP-style explanations [16], which the project already plans for the yield stage.

The comparative study remains useful: the LSTM shows that temperature, humidity and dew point can be forecast three days ahead with high accuracy, and these variables are available as supporting inputs if a later stage needs them.

5.8 Summary

A per-district, per-horizon two-stage hurdle model is the main forecasting model of the project. It forecasts daily rainfall 1–3 days ahead using only information available on the forecast day, with test R^2 of 0.15–0.29 one day ahead, falling to 0.07–0.15 at three days; its rain gate

classifies 81–88% of days correctly as dry or rainy. It gives better next-day rainfall forecasts than the LSTM, the comparative model evaluated under the same protocol. A comparative study of LSTM, SARIMA, XGBoost and TFT confirmed that models treating rainfall as a single continuous value struggle with its many dry days and rare heavy events, while the LSTM forecasts smoother variables such as temperature and humidity very well.

Chapter 6

Knowledge Graph from the Contingency Plans

6.1 Why a knowledge graph

The contingency plans contain exactly the guidance the system must give, but in a form a computer cannot use: tables of free text spread over 30 to 50 pages, with different layouts in different plans. The recommendations also depend on several things at once. The right action for a delayed monsoon depends on the district, the crop, the soil, the season and sometimes the crop stage, and soil types and crops are shared between districts.

A knowledge graph stores such facts as nodes (things) joined by typed edges (relations). It answers multi-hop questions that a flat lookup table cannot, for example "which past seasons in Beed had the same mid-season dry spell, and what soybean yield followed?", and it can keep, on every edge, the plan page the fact came from. The graph needs no data beyond what the preprocessing already produces, and at under a thousand nodes it is small enough to run in memory with NetworkX [28] and to export to a graph database such as Neo4j.

6.2 Build pipeline

The graph is built by five scripts, run in order by `run_all.py`:

Table 6.1: Knowledge-graph build steps

Step	Script	Output
1	`extract_pdfs.py`	District profile table; 1,063 condition→action rows; 27 "not applicable" statements
2	`label_seasons.py`	124 condition labels for past seasons; weather summary for all 260 district-seasons
3	`build_graph.py`	The graph, saved as JSON and GraphML, with summary statistics
4	`build_html_viewer.py`	An interactive viewer that opens in any browser
5	`export_neo4j.py`	One CSV per node and relation type plus an `import.cypher` script

6.3 Extracting the contingency tables

The extractor uses pdfplumber to read the tables page by page. The plans follow a common ICAR template but are not uniform, so the extractor reads each table's own header instead of assuming fixed columns. The main difficulties and how they were handled:

- **Different column meanings by table type.** Delayed-onset tables have *change in crop / agronomic measures / remarks*; normal-onset and mid-season tables have *crop management / soil nutrient and moisture conservation / remarks*; terminal-drought tables have *crop management / Rabi crop planning / remarks*; and the unusual-rain, flood and extreme-event tables have one column per crop stage (vegetative, flowering, maturity, post-harvest).
- **Wide and two-row headers.** Solapur's tables have 8 to 11 columns with spanned header cells, which are inherited from the left; its *Kharif* | *Rabi* sub-columns become separate rows, each with its own season. Beed uses two header rows; each column takes its role from whichever row names it.
- **Page breaks.** Text that spills onto the next page is appended to the row it belongs to, and split phrases such as "Delay by 4 weeks" + "2nd week of July" are re-joined.
- **Irrigated situations** (canal, tank, groundwater shortages) are given their own category instead of inheriting the drought heading printed above them.
- **Crop names inside free text,** e.g. "Kharif Pearlmillet Preseasonal sugarcane Maize", are found by scanning against the crop vocabulary; a "Rabi" or "Kharif" prefix sets the season.
- **"-do-" and "As above"** cells are resolved from the row above in the same column, and every resolution is recorded. Five dittos have nothing above them and are left flagged.

Accuracy check. The first 20 characters of every extracted field were searched for in the cells of the page it came from: of 4,027 fields, 99% were found verbatim, and the rest are the deliberate joins described above. Beed page 19 and Solapur page 11 were also compared visually with the PDF.

Each recommendation also needs a season. It is taken, in order of preference, from the PDF text itself ("Rabi sorghum", or a Kharif/Rabi sub-column), from the crop calendar for single-season crops (Soybean → Kharif, Gram → Rabi), or from the fact that Section 2.1.1 rainfed drought tables concern the Kharif monsoon. Where none applies the season is marked *unspecified*, and such rows are still returned by queries rather than silently dropped.

6.4 Season labeller

To connect the plans with real history, every past district-season from 2000 to 2025 is labelled with the conditions it actually experienced, using the plans' own definitions applied to the observed daily rainfall (Table 6.2).

Table 6.2: Rules used by the season labeller

Label	**Rule**
Onset delayed 2/4/6/8 weeks	Onset = first day from 1 June with a 3-day total $\geq$ 20 mm and no run of $\geq$ 14 rainless days ($<$ 2.5 mm) in the next 30 days. Delay is counted in weeks after 8 June, because the plans equate a 2-week delay with the 4th week of June.
Normal onset, then dry spell	Onset not delayed, and a run of $\geq$ 15 rainless days starting within 30 days of onset.
Mid-season dry spell (vegetative / flowering)	A run of $\geq$ 14 rainless days starting 31–60 / 61–90 days after onset.
Terminal drought	Rainfall from 15 September to 15 October below 50% of the district's own 2000–2025 mean for that window.
Continuous high rainfall	A 3-day total $\geq$ 150 mm (Kharif), or $\geq$ 64.5 mm in November–February (unseasonal, Rabi).

The labeller found 124 condition labels across the 260 district-seasons (Figure 6.1). The labels agree with known bad years: terminal drought appears in 2014 and 2018, the onset is delayed by about 8 weeks in 2000 and 2015, and Solapur's onset is delayed in 2012. For Beed, for example, 2018 is labelled *normal onset followed by a 25-day dry spell* and *terminal drought* (only 29.7 mm between 15 September and 15 October against a district mean of 141.1 mm), and that season's Kharif rainfall was 37% below the mean.

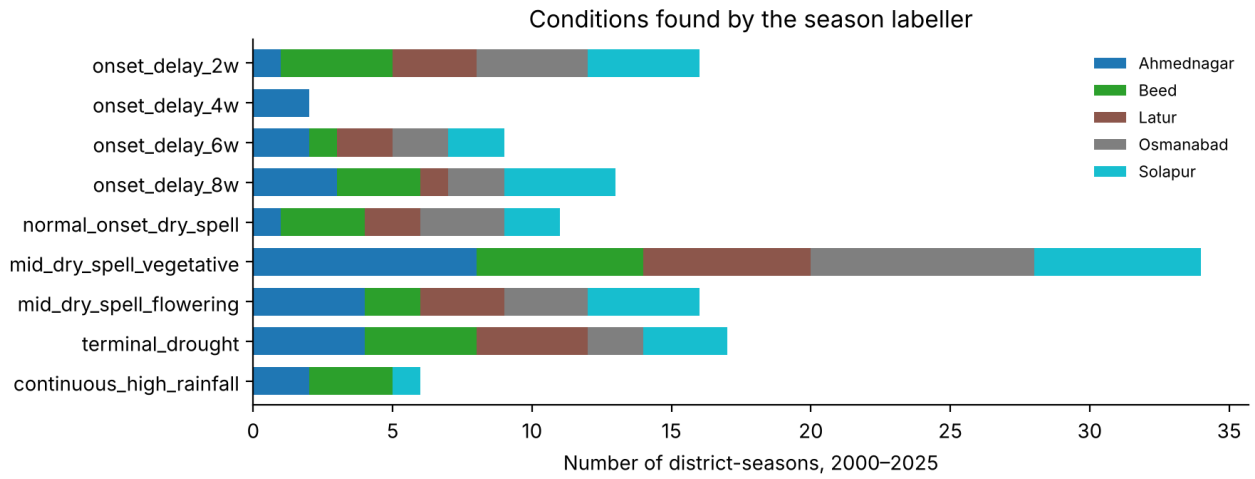


Figure 6.1: Number of district-seasons, 2000–2025, labelled with each condition. Mid-season dry spells during vegetative growth are the most common.

The labeller is a transparent first version, not a validated drought index, and it also acts as a stand-in for Stage 4: when Stage 4 is built, it will write a file with the same columns and the graph will use it without any change to its schema.

6.5 Graph schema

The finished graph has **833 nodes** of nine types and **4,353 edges** of fifteen relation types (Tables 6.3 and 6.4).

Table 6.3: Node types

Type	Count	What it represents
District	5	The district, with its rainfall normal and irrigated share from the plan
SoilType	12	Soil categories from the plans (e.g. medium-deep black soil)
Season	2	Kharif, Rabi
Crop	35	5 core, 4 extended, 26 fallback/horticulture crops named in the plans
Condition	7	Early-season, mid-season and terminal drought; irrigated water shortage; unusual rains; floods; extreme events
ConditionDetail	24	Precise sub-types with machine-readable values, e.g. onset delayed 4 weeks, 2-week dry spell at flowering, hailstorm
Action	462	Unique recommendation texts, split into the plan's own columns
Year	26	2000 to 2025
SeasonRecord	260	One per district, season and year, with that season's weather summary

Table 6.4: Main relation types

Relation	From → To	Count	Meaning
RECOMMENDS	ConditionDetail → Action	1,063	One per plan table row; carries district, crop, season, soil, crop stage and **source page**
APPLIES_TO_CROP	Crop → Action	702	The action is for this crop
SOIL_CONTEXT	SoilType → Action	399	The action is for this soil
SUGGESTS_CROP	Action → Crop	321	An alternative or Rabi follow-up crop named in the action
HAD_YIELD	SeasonRecord → Crop	788	The DES area, production and yield that season
EXPERIENCED	SeasonRecord → ConditionDetail	124	The condition the season actually had (from the labeller)
GROWS	District → Crop	84	The district grows this crop
NOT_APPLICABLE	District → Condition	27	The plan rules this condition out (e.g. floods in Beed)
HAS_SOIL	District → SoilType	15	With area and share

Others: SUBTYPE_OF (24), IN_SEASON (20), INCLUDES (6), HAS_SEASON_RECORD, IN_YEAR and OF_SEASON (260 each).

The core of the graph is the path

$$\text{Condition} \leftarrow \text{ConditionDetail} \xrightarrow{\text{RECOMMENDS}} \text{Action} \leftarrow \text{Crop},\ \text{SoilType}$$

filtered by the district and season stored on each RECOMMENDS edge. Around it, the SeasonRecord nodes link the official guidance to 26 years of observed history and yield. Figure 6.2 shows the interactive viewer.

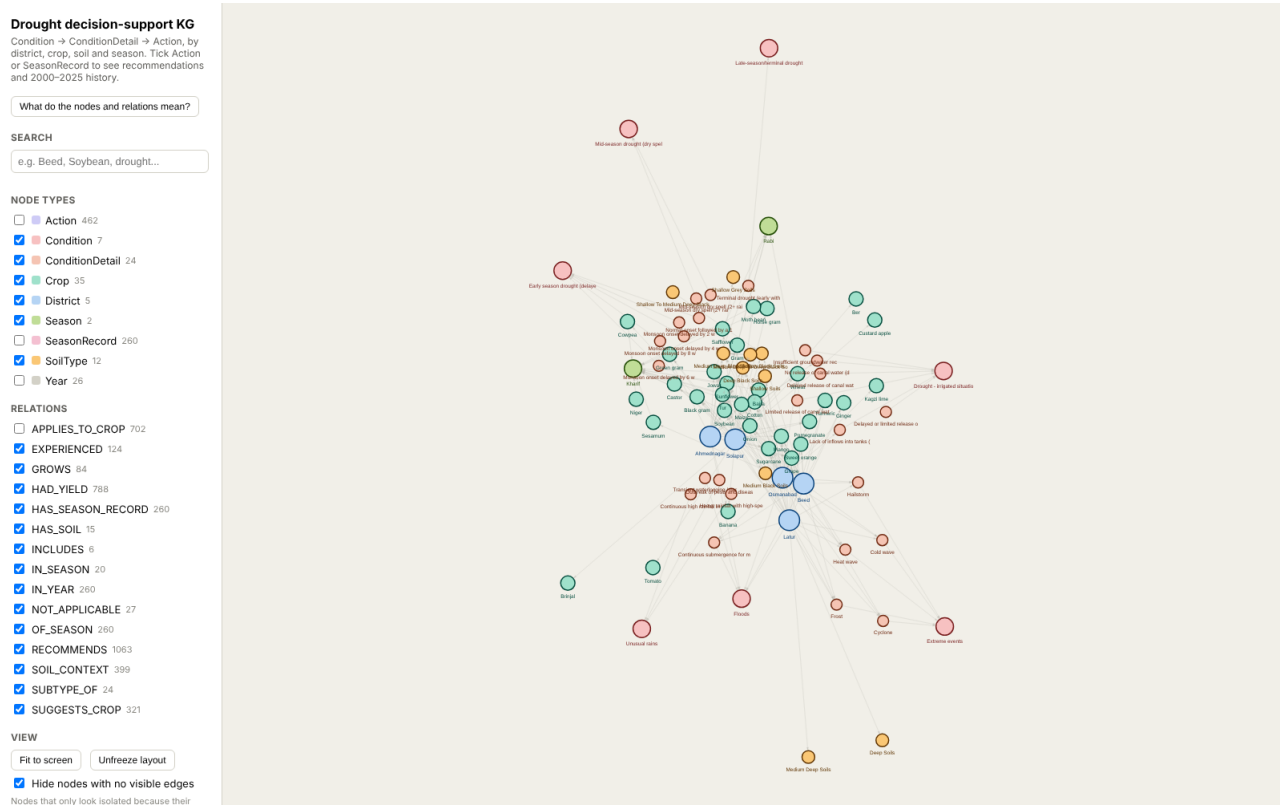


Figure 6.2: The interactive knowledge-graph viewer. Node types and relations can be switched on and off in the sidebar; clicking a node or edge shows its details and source. Actions and season records are hidden by default for readability.

6.6 Querying the graph

`kg_query.py` provides query functions (`recommend`, `alternatives`, `not_applicable`, `district_profile`, `season_history`, `similar_seasons`, `yield_baseline`) and one combined call designed for the LLM layer:

```
retrieve_context(G, district, crop, season, condition)
```

It returns a single JSON-ready bundle containing the condition's definition, the district profile, the official recommendations with page citations, alternative crops, past seasons with the same condition and the yields that followed, the all-years yield baseline, the plan's "not applicable" statements, and the guardrail text the LLM must obey. The condition can be given as a detail code (`onset_delay_4w`), a category code (`mid_season_drought`) or a category name.

6.6.1 Worked example

For *Beed, Soybean, Kharif*, the graph returns, among others, the following recommendations from the Beed plan:

Table 6.5: Selected Beed recommendations for soybean returned by the graph

Condition	Recommended action	Page
Onset delayed 2 weeks (4th week of June)	No change in cropping	9
Onset delayed 4 weeks (2nd week of July)	Soybean (JS-335, MAUS-71) + pigeon pea (BSMR-736, 853, BDN-708, 711) intercrop in 4:2 or 6:3 rows	10
Onset delayed 6 weeks (4th week of July)	Shift to pearl millet + pigeon pea in 4:2 or 3:3 rows	12
Onset delayed 8 weeks (2nd week of August)	Pearl millet or sunflower (e.g. Morden, SS-56, LSH-36)	13–14
Normal onset then 15–20 day dry spell	Gap-fill with the same or a short-duration cultivar to keep at least 75% plant population	15
Mid-season dry spell, vegetative	Interculture for weeding and to create a soil mulch; re-sow if plant population is below 50%	17–18
Mid-season dry spell, flowering	Give protective irrigation with sprinkler	18
Continuous high rainfall	Drain out excess water; after harvest, shift produce to a safe place and dry it	24

Alongside these, the graph returns Beed's soil profile (58.9% shallow black soils, 16% net irrigated) and the past Kharif seasons labelled with the same condition together with the soybean yield DES recorded for them. This is exactly the material the LLM will be allowed to use.

6.7 Neo4j export

`export_neo4j.py` writes one CSV per node and relation type and an `import.cypher` script that loads them, giving every node a unique key. It can also push the graph directly into a running database. A Cypher query template for the GraphRAG step is included. The export has been generated but not yet loaded into a live Neo4j instance; this is planned together with the GraphRAG layer.

6.8 Limitations

- **The plans date from 2011.** Soil, irrigation and rainfall-normal figures are a single snapshot; Ahmednagar's plan gives no soil percentages and Latur's net irrigated share is not captured.
- **Soil vocabularies differ** between a plan's profile section and its tables, so links between them are made only where names match exactly; several soil groups still need manual checking.

- **Some rows are not captured,** for example rows whose crop cell holds no recognised crop (grasses, tree plantations).
- **Mixed rotation cells** such as "Kharif fallow – Rabi sorghum / safflower / sunflower" can tag a crop with the wrong season.
- **The labeller uses gridded rainfall,** which smooths extremes, so heavy-rain labels are conservative; it cannot detect pests, hail or canal releases, which therefore come only from the plans.
- **Yield links are missing** for Gram, Wheat, Cotton and Safflower, which are absent from the DES table used.

6.9 Summary

The five contingency plans have been converted into a knowledge graph of 833 nodes and 4,353 edges in which every recommendation keeps its source page. A season labeller ties 26 years of observed weather and DES yields to the plans' own condition definitions, and a single query call returns everything the LLM layer will need to give a grounded, cited answer.

Chapter 7

MLOps Pipeline

7.1 Why MLOps

A forecasting model trained once and left alone slowly goes stale: new weather arrives every day, the climate shifts (rainfall in all five districts has risen over the 26 years, Chapter 3), and nobody notices when accuracy drops. Sculley et al. [33] point out that in real machine-learning systems the model is a small part of the whole, and most of the effort goes into data handling, configuration, serving and monitoring. MLOps is the practice of treating those parts as carefully as the model. For this project it means four things: the data and models are versioned, there is one clearly marked production model per district, forecasts are served through a stable interface, and performance is checked automatically.

7.2 Architecture

Figure 7.1 shows the flow. The pipeline is built around the project's main forecasting model, the per-district hurdle rainfall model; the comparative models are kept as research notebooks and are not served.

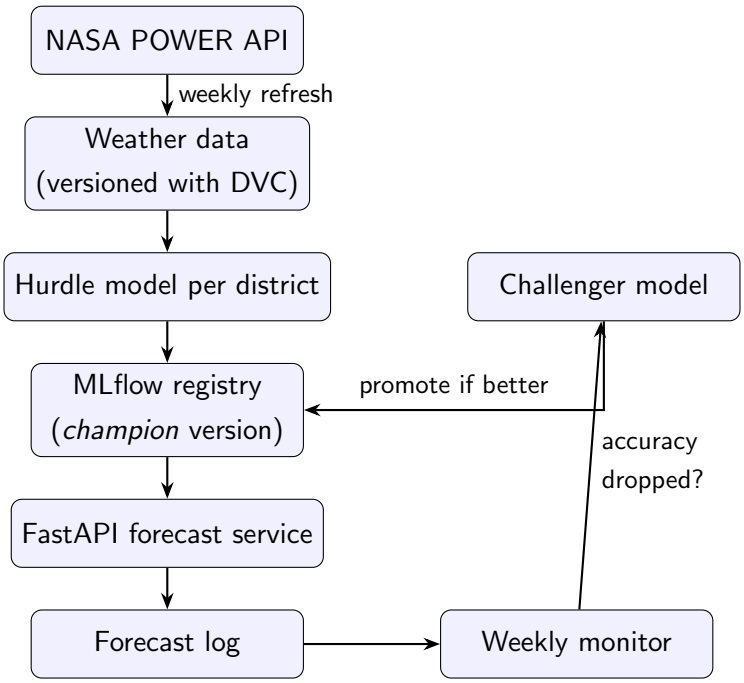


Figure 7.1: MLOps flow for the forecasting stage

7.3 Components

Table 7.1: MLOps components

Component	File / tool	What it does
Configuration	`params.yaml`	One place for districts, data paths, model settings (horizon 3, thresholds 0.25 mm and 0.35), split dates and monitoring thresholds
Data refresh	`fetch_nasa.py`, `clean.py`	Download the latest 60 days from NASA POWER and clean them into the weather folder
Data versioning	DVC with a DagsHub remote	Keeps every version of the data and model outputs outside Git, with a small pointer file in Git
Model pipeline	`hurdle_pipeline.py`	Feature engineering, leakage-safe per-horizon training and evaluation, forecasting
Registry	`registry.py`, MLflow	Logs each model with its parameters and metrics, versions it, and marks one version per district as the *champion*
Serving	`serve/app.py`, FastAPI, Docker	`/health` and `/forecast/{district}` endpoints; every forecast is appended to a log
Monitoring	`monitor.py`	Scores logged forecasts against observed rain and, if accuracy has dropped, trains and tests a challenger
Automation	GitHub Actions	CI tests on every push; weekly data refresh; weekly monitoring
Tests	`pytest`	Check feature engineering and the hurdle model contract

7.4 How it works

7.4.1 Data versioning

The weather files are too large and change too often to keep in Git. DVC (Data Version Control) stores them in a remote store on DagsHub and keeps only a small pointer file in Git. Every model can therefore be traced to the exact data it was trained on, and any earlier state can be restored with `dvc pull`.

7.4.2 Model registry and the champion

MLflow [29] records every trained model together with its parameters (district, horizon, test start date, probability threshold) and test metrics. Each district has one registered model, `<district>_hurdle`, with numbered versions. One version carries the alias *champion*; the

API always serves the champion, so switching to a better model means moving the alias rather than editing code.

7.4.3 Serving

The FastAPI service exposes `GET /health` and `GET /forecast/{district}`. On a request it fetches the district's champion from MLflow (and caches it), loads the latest weather, returns the forecast as JSON, and appends a line to `forecasts_log.csv` with the model version, the date the forecast was made, the target date and the predicted rainfall.

7.4.4 Automation

Three GitHub Actions workflows run without manual effort:

- **ci**: on every push or pull request, install the dependencies and run the tests;
- **weekly-data**: every Monday at 03:00 UTC (08:30 IST), pull the current data, fetch new NASA POWER data, clean it, push the new version to DVC and commit the updated pointer;
- **weekly-monitor**: every Monday at 06:00 UTC (11:30 IST), score the logged forecasts.

7.4.5 Monitoring and champion–challenger promotion

Once at least 30 days of logged forecasts can be compared with observed rainfall, the monitor computes RMSE and R^2 over the last 60 days. If R^2 has fallen more than 0.10 below the champion's recorded test R^2, it trains a *challenger* on the latest data, evaluates both models on the same most recent year, and promotes the challenger only if it is better. This keeps a worse model from replacing a good one by accident.

7.5 Current status and honest limitations

- The pipeline, registry, API, workflows and tests are in place and the five existing hurdle checkpoints have been registered.
- The registered checkpoints are the *original* hurdle models, from the first, same-day version of the model (Section 5.4.4). The API therefore labels its output `original_checkpoint_same_day`. Registering the corrected three-horizon models described in Chapter 5, already implemented in `hurdle_pipeline.py`, is the first task of the next phase.
- The monitor needs the forecast log to be versioned with DVC before the scheduled workflow can read it; this is a manual step today.

- The comparative models (LSTM, SARIMA, XGBoost, TFT) are not registered or served; if a later stage needs forecasts of other variables such as temperature, the LSTM can be added to the same pipeline.

Chapter 8

Conclusion and Scope for further Research

8.1 Conclusion of the first phase

This thesis set out to build a decision-support system that links weather, crop yield, human impact and official guidance for five drought-prone districts of Maharashtra, with every recommendation traceable to the District Agriculture Contingency Plans. In the first phase the team:

1. audited, cleaned and standardised four public data sources and documented their gaps;
2. built the main forecasting model, a per-district two-stage hurdle model that forecasts daily rainfall one to three days ahead using only information available on the forecast day (R^2 0.15 to 0.29 at one day, 0.07 to 0.15 at three days), after finding and correcting a data-leakage problem in its first version;
3. carried out a comparative study with LSTM, SARIMA, XGBoost and TFT models; the hurdle model gave better next-day rainfall forecasts than the LSTM in every district, while the LSTM forecast temperature, humidity and dew point well (R^2 0.85 to 0.94);
4. turned the five contingency plans into a knowledge graph of 833 nodes and 4,353 edges, with 1,063 cited recommendations, 26 years of labelled season history and DES yields, and a single query call for the LLM layer; and
5. set up an MLOps pipeline with data versioning, a model registry, a forecast API, weekly data refresh and monitoring with champion–challenger promotion.

Three of the ten stages (selection, forecasting and knowledge-graph retrieval) are complete, and the interfaces between stages are fixed, so the remaining stages can be built in parallel and plugged in.

8.2 Scope for further research: plan for the second phase

The second phase completes the remaining seven stages and turns the components into one working system. The plan below follows the pipeline order.

8.2.1 Strengthening the forecasting stage

- **Deploy the corrected hurdle model.** Register the three-horizon hurdle models in MLflow in place of the first, same-day checkpoints, and serve them through the API.
- **Improve the hurdle model.** Tune the rain-probability threshold and boosting settings, test the physically motivated features (vapour pressure deficit, dew-point spread) in the leakage-safe set-up, and add regional or large-scale predictors.
- **Longer outlook.** Because Stages 3–4 judge a whole season, investigate longer horizons: direct 7- to 14-day models, combining the in-house forecast for days 1–3 with an external forecast for later days, and season-level outlooks using large-scale climate indices.
- **Complete the comparative study** by re-running SARIMA, XGBoost and TFT under the same protocol as the hurdle model and the LSTM.

8.2.2 Stage 3: season aggregation

For each district, season and year, summarise the daily weather (observed so far plus forecast) into season-level features: total and monsoon rainfall, departure from the district mean, onset date and delay, longest dry spell, number of rainy days, maximum 3-day rainfall, and rainfall and heat during sensitive crop stages (sowing, vegetative, flowering). The season summary already produced for the knowledge graph will be extended rather than rebuilt.

8.2.3 Stage 4: condition classification

Classify each season against the district's own history using the Standardised Precipitation Index [18] at one- to three-month scales and tercile thresholds, combined with the plans' agronomic definitions already encoded in the season labeller. The output will use the same columns as the labeller's file, so the knowledge graph picks it up unchanged. Labels will be checked against known drought years (2014, 2015, 2018).

8.2.4 Stage 5: yield prediction

- **Target:** yield (t/ha) for each district, crop and season, from DES.
- **Features:** Stage-3 season aggregates, the district's soil and irrigation profile, and a time trend to capture improvements in seeds and practices.

- **Models:** a linear baseline (ElasticNet) and tree ensembles (random forest [15], gradient boosting). Deep learning is ruled out because there are only about 23 seasons per district-crop.
- **Validation:** rolling-origin or leave-one-year-out, so a year is always predicted from other years; results compared against a simple baseline (district trend or mean yield).
- **Data gap:** source Rabi Gram yields from the DES portal before modelling.

8.2.5 Stage 6: explanation

Apply SHAP [16] (TreeExplainer for tree models) to each yield prediction, so the system can state which factors, such as a long August dry spell or a late onset, pushed the predicted yield up or down, and by how much.

8.2.6 Stage 7: human impact

Convert the predicted shortfall into impact: production loss = (expected yield − predicted yield) × cropped area; then relate it to the district's population and agricultural workforce from Census 2011. To express the loss as food need, a per-capita consumption norm (for example from ICMR-NIN dietary guidelines or Public Distribution System allocations) still has to be sourced.

8.2.7 Stage 8: district comparison

Normalise a severity score (yield shortfall, SPI) and an exposure score (population, workers) to a common 0–1 scale, combine them into a priority index, and rank the five districts. The sensitivity of the ranking to the weights will be reported.

8.2.8 Stages 9–10: knowledge graph and guarded GraphRAG

- Load the graph into Neo4j, check the remaining soil-vocabulary links by hand, and add the extended crops.
- Build the GraphRAG layer: an LLM extracts district, crop, season and condition from a question; templated Cypher queries retrieve the matching subgraph through `retrieve_context()`; the LLM writes one answer from the retrieved facts and Stage 5–8 numbers only.
- Enforce the guardrail in code: every number in the answer must appear in the retrieved context, and every recommendation must carry a plan page. Evaluate on a set of test questions for citation correctness and faithfulness.

8.2.9 Integration, interface and evaluation

- A Streamlit interface where a user picks a district and season (or types a question) and sees the forecast, the season's condition, the predicted yield with its SHAP explanation, the human impact, the district ranking and the cited official actions.
- End-to-end hindcast case studies on known drought years (e.g. Beed 2018, Osmanabad 2015) to check that the system would have flagged the right conditions and returned the right guidance.
- Final report and a research paper.

8.2.10 Timeline

Table 8.1: Plan for the second phase (eighth semester)

Weeks	Work	Lead
1–3	Leakage-safe models into production; common benchmark protocol; source Gram data and consumption norms	Gowtham, Guhan, Vaishnavi, Vyshnav
3–6	Season aggregation (Stage 3) and condition classification (Stage 4); forecasts fed into aggregation	Guhan, Vyshnav
5–9	Yield model (Stage 5) and SHAP explanations (Stage 6)	Guhan, Vaishnavi
8–11	Human impact (Stage 7) and district ranking (Stage 8)	Vaishnavi
6–11	Neo4j load and graph refinement; GraphRAG layer with guardrail (Stage 10)	Vyshnav, Gowtham
11–13	Integration, Streamlit interface, MLOps for all deployed models	All
13–15	Hindcast case studies, testing, final report and paper	All

8.3 Closing remarks

The contingency plans already contain the expert knowledge that farmers and officials need during a drought; what has been missing is a way to connect that knowledge to the weather and the crop as they unfold. The first phase has built the two ends of that bridge: a forecasting stage that reads each district's own weather history, and a knowledge graph that makes the official guidance queryable and citable. The second phase builds the middle, from season

condition through yield and impact to a plain-language answer that a district officer can trust because every part of it can be checked.

Appendix A

Configuration, Schema and Interface Details

A.1 Main model: hurdle configuration (params.yaml)

Table A.1: Hurdle model and monitoring settings

Setting	Value
Rain/no-rain threshold	0.25 mm
Rain-probability threshold τ	0.35
Classifier	HistGradientBoosting, 250 iterations, learning rate 0.04
Regressor	HistGradientBoosting on $\log(1+y)$, 350 iterations, learning rate 0.03
L2 regularisation	0.1
Test start / validation	1 Jan 2022 / last 365 days before test
Monitor window / minimum days	60 / 30 days
R^2 drop tolerance	0.10
NASA refresh window	60 days

A.2 Comparative model: LSTM configuration

The same configuration is used for all five districts (stored as `config.json` in each district's folder under `outputs/plots/lstm`).

Table A.2: LSTM hyper-parameters

Setting	Value
Lookback window	45 days
Forecast horizon	3 days (direct multi-step)
Input features	19 per day (11 raw + 8 rolling) + day-of-year sin/cos
Output	11 variables × 3 days
Log-transformed columns	Rainfall, wind speed
LSTM layers / units	1 / 128
Dense layers	384, 192 units; dropout 0.25
Loss	Huber
Optimiser / learning rate	Adam / 5×10^{-4}
Batch size	16
Early stopping	Patience 12 epochs on validation loss
Training / validation / test	to 31-12-2018 / 2019–2021 / 2022 onward

A.3 Comparative LSTM: per-variable test R^2 for all districts

Table A.3: Test R^2 by district and variable

Variable	Ahmednagar	Beed	Latur	Osmanabad	Solapur
Rainfall	0.101	0.209	0.218	0.163	0.169
Mean temperature	0.922	0.932	0.929	0.922	0.917
Max temperature	0.886	0.892	0.890	0.878	0.859
Min temperature	0.896	0.908	0.903	0.897	0.888
Relative humidity	0.937	0.934	0.932	0.927	0.927
Wind speed	0.718	0.699	0.694	0.676	0.728
Surface pressure	0.835	0.851	0.803	0.809	0.814
Solar radiation	0.650	0.571	0.529	0.530	0.545
Dew point	0.900	0.892	0.878	0.854	0.871
Cloud cover	0.747	0.733	0.724	0.722	0.717
Wind speed at 50 m	0.641	0.632	0.615	0.592	0.660
Blended	**0.749**	**0.750**	**0.738**	**0.725**	**0.736**

A.4 Knowledge-graph interface

- Build everything: `python codes/knowledge_graph/run_all.py`
- List condition codes:
  `python codes/knowledge_graph/kg_query.py --list-conditions`

- Example query:
  `kg_query.py --district Beed --crop Soybean --condition mid_dry_spell_flowering`
- Python call for the LLM layer: `retrieve_context(G, district, crop, season, condition)`
- Node keys have the form `<Type>:<name or code>`; relations are read from the `relation` attribute of each edge.

Condition-detail codes used by the labeller: `onset_delay_2w`, `onset_delay_4w`, `onset_delay_6w`, `onset_delay_8w`, `normal_onset_dry_spell`, `mid_dry_spell_vegetative`, `mid_dry_spell_flowering`, `terminal_drought`, `continuous_high_rainfall`.

A.5 Forecast API

Table A.4: FastAPI endpoints

Endpoint	Response
`GET /health`	{`status: ok`}
`GET /forecast/{district}`	District, champion model version, origin date, forecast type, and a list of (target date, rainfall in mm). Unknown districts return HTTP 404; a missing model or data returns HTTP 503.

A.6 Repository layout

- `codes/preprocessing.ipynb` — downloads and cleaning
- `codes/forecasting/` — hurdle pipeline (main model), registry, monitor, comparative LSTM notebook
- `codes/knowledge_graph/` — extraction, labeller, graph build, viewer, Neo4j export, queries
- `codes/pipeline/` — scheduled NASA refresh and cleaning
- `serve/` — FastAPI service; `tests/` — pytest suite
- `data/`, `models/`, `outputs/` — data, weights, results (large files tracked with DVC)
- `MLOps/` — implementation notes and operations runbook

References

[1] P. Udmale, Y. Ichikawa, S. Manandhar, H. Ishidaira, and A. S. Kiem, "Farmers' perception of drought impacts, local adaptation and administrative mitigation measures in Maharashtra state, India," *International Journal of Disaster Risk Reduction*, vol. 10, pp. 250–269, 2014.

[2] ICAR – Central Research Institute for Dryland Agriculture, "District agriculture contingency plans, Maharashtra: Ahmednagar, Beed, Latur, Osmanabad and Solapur." ICAR-CRIDA, Hyderabad, and Ministry of Agriculture, Government of India, 2011.

[3] Z. Ji, N. Lee, R. Frieske, *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[4] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. John Wiley & Sons, 5th ed., 2015.

[5] S. Mulla, C. B. Pande, and S. K. Singh, "Time series forecasting of monthly rainfall using seasonal auto regressive integrated moving average with exogenous variables (SARIMAX) model," *Water Resources Management*, vol. 38, pp. 1825–1846, 2024.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[7] M. Dehvari, S. Farzaneh, and E. Forootan, "Forecasting rainfall events based on zenith wet delay time series utilizing eXtreme gradient boosting (XGBoost)," *Advances in Space Research*, vol. 75, no. 3, pp. 2584–2598, 2025.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] S. N. H. Bukhari and K. A. Ogudo, "Forecasting temperature and rainfall using deep learning for the challenging climates of Northern India," *PeerJ Computer Science*, vol. 11, p. e3012, 2025.

[10] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.

[11] J. G. Cragg, "Some statistical models for limited dependent variables with application to the demand for durable goods," *Econometrica*, vol. 39, no. 5, pp. 829–844, 1971.

[12] J. Mullahy, "Specification and testing of some modified count data models," *Journal of Econometrics*, vol. 33, no. 3, pp. 341–365, 1986.

[13] R. D. Stern and R. Coe, "A model fitting analysis of daily rainfall data," *Journal of the Royal Statistical Society: Series A (General)*, vol. 147, no. 1, pp. 1–34, 1984.

[14] T. van Klompenburg, A. Kassahun, and C. Catal, "Crop yield prediction using machine learning: A systematic literature review," *Computers and Electronics in Agriculture*, vol. 177, p. 105709, 2020.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[16] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[17] A. K. Mishra and V. P. Singh, "A review of drought concepts," *Journal of Hydrology*, vol. 391, no. 1–2, pp. 202–216, 2010.

[18] T. B. McKee, N. J. Doesken, and J. Kleist, "The relationship of drought frequency and duration to time scales," in *Proceedings of the 8th Conference on Applied Climatology*, (Anaheim, CA), pp. 179–184, 1993.

[19] A. Hogan, E. Blomqvist, M. Cochez, *et al.*, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

[20] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[21] D. Edge, H. Trinh, N. Cheng, *et al.*, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[22] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.

[23] NASA Langley Research Center, "NASA prediction of worldwide energy resources (POWER) project." `https://power.larc.nasa.gov`. Daily data, 2000–2025.

[24] Directorate of Economics and Statistics, Ministry of Agriculture and Farmers Welfare, Government of India, "District-wise crop production statistics." `https://data.desagri.gov.in`. 2000–01 to 2022–23.

[25] Office of the Registrar General and Census Commissioner, India, "Census of India 2011: District-wise population and workers." `https://censusindia.gov.in`, 2011.

[26] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[27] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[28] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pp. 11–15, 2008.

[29] M. Zaharia, A. Chen, A. Davidson, *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[30] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[31] P. J. Huber, "Robust estimation of a location parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[32] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[33] D. Sculley, G. Holt, D. Golovin, *et al.*, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.